```
# ========================================================================
# 文件: api/v1/reagents.py  (D:\workbuddyprojects\网页版-生免速查工具\backend\app\api\v1\reagents.py)
# ========================================================================

from ...core.crud_base import make_router
from ...models.reagent import Reagent
from ...schemas import ReagentCreate, ReagentRead, ReagentUpdate

router = make_router(
    Reagent,
    ReagentRead,
    ReagentCreate,
    ReagentUpdate,
    search_fields=["name", "brand", "spec", "lot_no", "supplier", "operator"],
    filter_fields=["status", "brand"],
    prefix="/reagents",
    write_roles=("admin", "reagent_manager"),
)

# ========================================================================
# 文件: api/v1/reagent_management.py  (D:\workbuddyprojects\网页版-生免速查工具\backend\app\api\v1\reagent_management.py)
# ========================================================================

"""试剂管理：试剂目录/库存/盘库/订购/接收/月消耗 API"""

import re
from datetime import date, datetime
from decimal import Decimal
from typing import Optional
from collections import defaultdict

from io import BytesIO

from fastapi import APIRouter, Depends, HTTPException, Query, UploadFile, File
from fastapi.responses import StreamingResponse
from sqlalchemy import or_, func
from sqlalchemy.orm import Session
from pydantic import BaseModel

from ...core.crud_base import paginate
from ...core.database import get_db
from ...core.security import get_current_user, require_roles, user_roles_list
from ...models.reagent_management import (
    ReagentItem, ReagentStock, InventoryCheck, InventoryCheckItem,
    ReagentOrder, ReagentOrderItem, Receiving, ReceivingItem, ReagentConsumption,
    TestItemReagent, InstrumentReagent,
    detect_reagent_type, derive_library,
)
from ...models.test_item import TestItem
from ...models.instrument import Instrument
from ...models.instrument_family import InstrumentFamily, InstrumentFamilyMember
from ...models.user import User
from ...schemas.reagent_management import (
    ReagentItemCreate, ReagentItemUpdate, ReagentItemRead,
    ReagentStockRead, InventoryCheckCreate, InventoryCheckRead,
    ReagentOrderCreate, ReagentOrderRead, ReceivingCreate, ReceivingRead,
    ReagentConsumptionRead, ImportResult,
```

```
    TestItemReagentCreate, TestItemReagentUpdate, TestItemReagentRead,
    InstrumentReagentCreate, InstrumentReagentUpdate, InstrumentReagentRead,
)

router = APIRouter(prefix="/reagent", tags=["reagent-management"])


# ============================================================================
# 1. 试剂目录 CRUD
# ============================================================================

@router.get("/items", response_model=dict)
def list_reagent_items(
    q: str = Query("", description="搜索（名称/品牌/材料编码/项目名/项目别名）"),
    type: Optional[str] = Query(None, description="类型筛选"),
    category: Optional[str] = Query(None, description="类别筛选"),
    library: Optional[str] = Query(None, description="责任库筛选（生化凝血/免疫）"),
    show_inactive: bool = Query(False, description="显示停用项（默认仅启用）"),
    page: int = Query(1, ge=1),
    page_size: int = Query(50, ge=1, le=200),
    db: Session = Depends(get_db),
    _=Depends(get_current_user),
):
    base = db.query(ReagentItem)
    if q.strip():
        kw = f"%{q.strip()}%"
        # 通过关联表匹配项目名/项目别名（如 alt → 丙氨酸氨基转移酶）
        matched_reagent_ids = db.query(TestItemReagent.reagent_item_id).join(
            TestItem, TestItem.id == TestItemReagent.test_item_id
        ).filter(
            or_(TestItem.name.like(kw), TestItem.aliases.like(kw))
        ).distinct().subquery()
        base = base.filter(
            or_(
                ReagentItem.name.like(kw), ReagentItem.brand.like(kw),
                ReagentItem.material_code.like(kw), ReagentItem.spec.like(kw),
                ReagentItem.id.in_(matched_reagent_ids),
            )
        )
    if type:
        base = base.filter(ReagentItem.type == type)
    if category:
        base = base.filter(ReagentItem.category == category)
    if library:
        base = base.filter(ReagentItem.library == library)
    if not show_inactive:
        base = base.filter(ReagentItem.is_active == True)
    total = base.count()
    # 默认排序：按年用量降序（多的在前），空值排最后，再按名称
    rows = base.order_by(
```

```
        ReagentItem.annual_usage.is_(None),
        ReagentItem.annual_usage.desc(),
        ReagentItem.name,
    ).offset((page - 1) * page_size).limit(page_size).all()
    return {"total": total, "page": page, "page_size": page_size,
            "items": [ReagentItemRead.model_validate(r) for r in rows]}


@router.get("/items/counts", response_model=dict)
def get_reagent_item_counts(
    library: Optional[str] = Query(None, description="责任库筛选（生化凝血/免疫）"),
    active_only: bool = Query(True, description="仅统计启用项"),
    db: Session = Depends(get_db),
    _=Depends(get_current_user),
):
    """按类型统计试剂目录数量。用于试剂目录页/工作台展示：试剂/校准品/耗材/质控品。"""
    base = db.query(ReagentItem)
    if library:
        base = base.filter(ReagentItem.library == library)
    if active_only:
        base = base.filter(ReagentItem.is_active == True)
    rows = base.with_entities(ReagentItem.type, func.count()).group_by(ReagentItem.type).all()
    counts = {"试剂": 0, "校准品": 0, "耗材": 0, "质控品": 0}
    for t, n in rows:
        if t in counts:
            counts[t] = n
    return {
        "library": library,
        "active_only": active_only,
        "total": sum(counts.values()),
        "counts": counts,
    }


@router.get("/items/{item_id}", response_model=ReagentItemRead)
def get_reagent_item(item_id: int, db: Session = Depends(get_db), _=Depends(get_current_user)):
    item = db.query(ReagentItem).get(item_id)
    if not item:
        raise HTTPException(404, "试剂未找到")
    return item


@router.post("/items", response_model=ReagentItemRead)
def create_reagent_item(
    data: ReagentItemCreate, db: Session = Depends(get_db),
    user: User = Depends(require_roles("admin", "reagent_manager", "lab_technician")),
):
    item = ReagentItem(**data.model_dump())
    db.add(item)
    db.commit()
```

```
    db.refresh(item)
    return item


@router.put("/items/{item_id}", response_model=ReagentItemRead)
def update_reagent_item(
    item_id: int, data: ReagentItemUpdate, db: Session = Depends(get_db),
    user: User = Depends(require_roles("admin", "reagent_manager", "lab_technician")),
):
    item = db.query(ReagentItem).get(item_id)
    if not item:
        raise HTTPException(404, "试剂未找到")
    for k, v in data.model_dump(exclude_unset=True).items():
        setattr(item, k, v)
    db.commit()
    db.refresh(item)
    return item


@router.delete("/items/{item_id}")
def delete_reagent_item(
    item_id: int, db: Session = Depends(get_db),
    user: User = Depends(require_roles("admin")),
):
    item = db.query(ReagentItem).get(item_id)
    if not item:
        raise HTTPException(404, "试剂未找到")
    db.delete(item)
    db.commit()
    return {"ok": True}


class _UsageUpdate(BaseModel):
    item_id: int
    annual_usage: int


class _BatchUsageUpdate(BaseModel):
    updates: list[_UsageUpdate]


@router.post("/items/_batch-update-usage", response_model=dict)
def batch_update_usage(
    payload: _BatchUsageUpdate, db: Session = Depends(get_db),
    user: User = Depends(require_roles("admin", "reagent_manager", "lab_technician")),
):
    """批量写入试剂年用量（来自采购明细），用于目录/盘库/订购默认排序。"""
    n = 0
    for u in payload.updates:
        it = db.query(ReagentItem).get(u.item_id)
```

```
        if it:
            it.annual_usage = int(u.annual_usage)
            n += 1
    db.commit()
    return {"updated": n}


# ============================================================================
# 2. 实时库存
# ============================================================================

@router.get("/stock", response_model=dict)
def list_stock(
    q: str = Query("", description="搜索试剂名称/品牌"),
    type: Optional[str] = Query(None),
    library: Optional[str] = Query(None, description="责任库筛选（生化凝血/免疫）"),
    low_stock_only: bool = Query(False, description="只显示低于最低库存预警的"),
    page: int = Query(1, ge=1),
    page_size: int = Query(50, ge=1, le=200),
    db: Session = Depends(get_db),
    _=Depends(get_current_user),
):
    base = db.query(ReagentStock)
    if q.strip():
        kw = f"%{q.strip()}%"
        item_ids = [r[0] for r in db.query(ReagentItem.id).filter(
            or_(ReagentItem.name.like(kw), ReagentItem.brand.like(kw))
        ).all()]
        base = base.filter(ReagentStock.item_id.in_(item_ids))
    if type:
        item_ids2 = [r[0] for r in db.query(ReagentItem.id).filter(ReagentItem.type == type).all()]
        base = base.filter(ReagentStock.item_id.in_(item_ids2))
    if library:
        item_ids3 = [r[0] for r in db.query(ReagentItem.id).filter(ReagentItem.library == library).all()]
        base = base.filter(ReagentStock.item_id.in_(item_ids3))
    if low_stock_only:
        # 仅显示低于最低库存预警的：join 试剂目录，按 min_stock>0 且 数量<min_stock 过滤
        base = base.join(ReagentItem, ReagentItem.id == ReagentStock.item_id).filter(
            ReagentItem.min_stock > 0,
            ReagentStock.quantity < ReagentItem.min_stock,
        )
    total = base.count()
    rows = base.order_by(ReagentStock.item_id).offset((page - 1) * page_size).limit(page_size).all()
    return {"total": total, "page": page, "page_size": page_size,
            "items": [ReagentStockRead.model_validate(r) for r in rows]}


# ============================================================================
# 3. 盘库
# ============================================================================
```

```
@router.get("/inventory-checks", response_model=dict)
def list_inventory_checks(
    library: Optional[str] = Query(None, description="责任库筛选（生化凝血/免疫）"),
    page: int = Query(1, ge=1),
    page_size: int = Query(20, ge=1, le=100),
    db: Session = Depends(get_db),
    _=Depends(get_current_user),
):
    base = db.query(InventoryCheck)
    if library:
        base = base.filter(InventoryCheck.library == library)
    total = base.count()
    rows = base.order_by(InventoryCheck.check_date.desc()).offset(
        (page - 1) * page_size
    ).limit(page_size).all()
    return {"total": total, "page": page, "page_size": page_size,
            "items": [InventoryCheckRead.model_validate(r) for r in rows]}


@router.get("/inventory-checks/{check_id}", response_model=InventoryCheckRead)
def get_inventory_check(check_id: int, db: Session = Depends(get_db), _=Depends(get_current_user)):
    check = db.query(InventoryCheck).get(check_id)
    if not check:
        raise HTTPException(404, "盘库未找到")
    return check


@router.delete("/inventory-checks/{check_id}", response_model=dict)
def delete_inventory_check(
    check_id: int, db: Session = Depends(get_db),
    user: User = Depends(require_roles("admin", "reagent_manager", "lab_technician")),
):
    """删除整次盘库记录（级联删除其细项）。"""
    check = db.query(InventoryCheck).get(check_id)
    if not check:
        raise HTTPException(404, "盘库未找到")
    db.query(InventoryCheckItem).filter(InventoryCheckItem.check_id == check_id).delete()
    db.delete(check)
    db.commit()
    return {"ok": True}


def _item_dto(ri: ReagentItem, stock_map: dict,
              project_name: str = "", project_aliases: str = "") -> dict:
    return {
        "item_id": ri.id, "name": ri.name, "spec": ri.spec, "type": ri.type,
        "unit": ri.unit, "material_code": ri.material_code or "",
        "brand": ri.brand or "",
        "current_stock": int(stock_map.get(ri.id, 0)), "library": ri.library,
```

```
        "min_stock": int(ri.min_stock or 0),
        "annual_usage": int(ri.annual_usage or 0),
        "project_name": project_name,
        "project_aliases": project_aliases,
    }


def _sort_items_by_usage(items: list) -> list:
    """组内试剂按年用量降序（多的在前），空值排最后，再按名称。"""
    return sorted(items, key=lambda it: (it.get("annual_usage", 0) == 0,
                                         -(it.get("annual_usage", 0) or 0),
                                         it.get("name", "")))


def _group_usage_key(group: dict) -> int:
    """组排序键：取组内最大年用量（空组排最后）。"""
    items = group.get("items") or []
    if not items:
        return 0
    return max((it.get("annual_usage", 0) or 0) for it in items)


# 仪器名关键词黑名单—这些仪器不产生耗材盘库条目（流水线等）
_NO_CONSUMABLE_INSTRUMENT_KEYWORDS = ("流水线",)


def build_reagent_template(db: Session, library: str) -> dict:
    """构建盘库/订购录入模板，按「项目 / 仪器家族 / 质控品」分组。

    规则：
    - by_project：通过 项目↔试剂 关联(role=试剂/校准品) 归组，_library 过滤，
      仅含在用试剂(is_active=True)；额外补充「未关联项目的试剂/校准品」。
    - by_instrument：通过 仪器↔耗材 关联(role=耗材) 归组，按仪器家族(总型号)合并
      （如 AU 生化仪 多台共用耗材只列一次），排除停用仪器与流水线；
      额外补充「未关联仪器的耗材」。
    - controls：type=质控品 且 library 匹配且 is_active（单独列出）。
    每个物品带 current_stock（实时库存合计）+ project_name/project_aliases（供前端模糊检索）。
    """
    stock_map = {}
    for iid, qty in db.query(
        ReagentStock.item_id, func.coalesce(func.sum(ReagentStock.quantity), 0)
    ).group_by(ReagentStock.item_id).all():
        stock_map[iid] = qty

    # ── 按项目（仅在用试剂）──
    tir_rows = (
        db.query(TestItemReagent, TestItem, ReagentItem)
        .join(TestItem, TestItem.id == TestItemReagent.test_item_id)
        .join(ReagentItem, ReagentItem.id == TestItemReagent.reagent_item_id)
        .filter(
```

```
            TestItemReagent.role.in_(["试剂", "校准品"]),
            ReagentItem.library == library,
            ReagentItem.is_active == True,
        )
        .order_by(TestItem.name, ReagentItem.name)
        .all()
    )
    proj_map = {}
    linked_reagent_ids = set()
    for tir, ti, ri in tir_rows:
        linked_reagent_ids.add(ri.id)
        key = ti.id
        if key not in proj_map:
            proj_map[key] = {"test_item_id": ti.id, "test_item_name": ti.name,
                             "test_item_aliases": ti.aliases or "", "items": []}
        proj_map[key]["items"].append(_item_dto(ri, stock_map, ti.name, ti.aliases or ""))
    # 未关联项目的试剂/校准品（仅在用）
    orphans = (
        db.query(ReagentItem)
        .filter(ReagentItem.type.in_(["试剂", "校准品"]),
                ReagentItem.library == library,
                ReagentItem.is_active == True)
        .order_by(ReagentItem.name).all()
    )
    orphan_items = [r for r in orphans if r.id not in linked_reagent_ids]
    if orphan_items:
        proj_map["__orphan__"] = {
            "test_item_id": None, "test_item_name": "未关联项目的试剂 / 校准品",
            "test_item_aliases": "", "items": [_item_dto(r, stock_map) for r in orphan_items],
        }
    by_project = list(proj_map.values())
    for g in by_project:
        g["items"] = _sort_items_by_usage(g["items"])
    by_project.sort(key=_group_usage_key, reverse=True)

    # ── 按仪器家族（仅在用仪器 + 非流水线 + 在用耗材）──
    ir_rows = (
        db.query(InstrumentReagent, Instrument, ReagentItem)
        .join(Instrument, Instrument.id == InstrumentReagent.instrument_id)
        .join(ReagentItem, ReagentItem.id == InstrumentReagent.reagent_item_id)
        .filter(
            InstrumentReagent.role == "耗材",
            ReagentItem.library == library,
            ReagentItem.is_active == True,
            Instrument.status != "停用",
        )
        .all()
    )
    # 过滤掉流水线类仪器
    ir_rows = [
```

```
        (ir, ins, ri) for ir, ins, ri in ir_rows
        if not any(kw in (ins.name or "") for kw in _NO_CONSUMABLE_INSTRUMENT_KEYWORDS)
    ]
    fam_members = db.query(InstrumentFamilyMember).all()
    fam_by_inst = {m.instrument_id: m.family_id for m in fam_members}
    fam_names = {f.id: f.name for f in db.query(InstrumentFamily).all()}
    fam_map = {}
    linked_consumable_ids = set()
    for ir, ins, ri in ir_rows:
        linked_consumable_ids.add(ri.id)
        fam_id = fam_by_inst.get(ins.id)
        if fam_id:
            gkey, gname = ("fam", fam_id), fam_names.get(fam_id, f"仪器组{fam_id}")
        else:
            gkey, gname = ("inst", ins.id), (ins.name or f"仪器{ins.id}")
        gkey_s = str(gkey)
        if gkey_s not in fam_map:
            fam_map[gkey_s] = {"group": gname, "instruments": [], "items": []}
        # 显示格式：名称(model)
        ins_label = ins.name
        if ins.model:
            ins_label = f"{ins.name}({ins.model})"
        if ins_label and ins_label not in fam_map[gkey_s]["instruments"]:
            fam_map[gkey_s]["instruments"].append(ins_label)
        if not any(it["item_id"] == ri.id for it in fam_map[gkey_s]["items"]):
            fam_map[gkey_s]["items"].append(_item_dto(ri, stock_map))
    # 未关联仪器的耗材（仅在用）
    cons_orphans = (
        db.query(ReagentItem)
        .filter(ReagentItem.type == "耗材", ReagentItem.library == library,
                ReagentItem.is_active == True)
        .order_by(ReagentItem.name).all()
    )
    cons_orphan_items = [r for r in cons_orphans if r.id not in linked_consumable_ids]
    if cons_orphan_items:
        fam_map["__cons_orphan__"] = {
            "group": "未关联仪器的耗材", "instruments": [],
            "items": [_item_dto(r, stock_map) for r in cons_orphan_items],
        }
    by_instrument = list(fam_map.values())
    for g in by_instrument:
        g["items"] = _sort_items_by_usage(g["items"])
    by_instrument.sort(key=_group_usage_key, reverse=True)

    # ── 质控品（单独，仅在用）──
    controls = [
        _item_dto(r, stock_map) for r in
        db.query(ReagentItem).filter(
            ReagentItem.type == "质控品", ReagentItem.library == library,
            ReagentItem.is_active == True,
```

```
        ).order_by(ReagentItem.name).all()
    ]
    controls = _sort_items_by_usage(controls)

    return {"by_project": by_project, "by_instrument": by_instrument, "controls": controls}


@router.get("/template", response_model=dict)
def get_reagent_template(
    library: str = Query("", description="责任库（生化凝血/免疫），为空返回全部"),
    db: Session = Depends(get_db),
    _=Depends(get_current_user),
):
    """盘库/订购录入模板：按项目、仪器家族、质控品分组，含实时库存。"""
    libs = [library] if library else ["生化凝血", "免疫"]
    result = {}
    for lib in libs:
        result[lib] = build_reagent_template(db, lib)
    return result


@router.post("/inventory-checks", response_model=InventoryCheckRead)
def create_inventory_check(
    data: InventoryCheckCreate, db: Session = Depends(get_db),
    user: User = Depends(require_roles("admin", "reagent_manager", "lab_technician")),
):
    check = InventoryCheck(
        library=data.library or "",
        check_date=data.check_date,
        check_type=data.check_type,
        operator=user.full_name or user.username,
        remark=data.remark,
    )
    db.add(check)
    db.flush()  # 获取 id
    for it in data.items:
        item = InventoryCheckItem(
            check_id=check.id, item_id=it.item_id, batch_no=it.batch_no,
            expiry_date=it.expiry_date, recorded_quantity=it.recorded_quantity,
        )
        db.add(item)
        # 同步更新 reagent_stock
        # 盘库按「总余量」录入：若未指定批号，视为对该物品做全量盘点，
        # 先清空该物品全部批次库存，再写入一条总余量，保证实时库存与盘库数一致。
        if it.batch_no and it.batch_no.strip():
            stock = db.query(ReagentStock).filter(
                ReagentStock.item_id == it.item_id,
                ReagentStock.batch_no == it.batch_no,
            ).first()
            if stock:
```

```
                stock.quantity = it.recorded_quantity
                stock.last_updated = datetime.utcnow()
            else:
                db.add(ReagentStock(
                    item_id=it.item_id, batch_no=it.batch_no,
                    expiry_date=it.expiry_date, quantity=it.recorded_quantity,
                ))
        else:
            for s in db.query(ReagentStock).filter(ReagentStock.item_id == it.item_id).all():
                db.delete(s)
            db.add(ReagentStock(
                item_id=it.item_id, batch_no="",
                expiry_date=it.expiry_date, quantity=it.recorded_quantity,
            ))
    db.commit()
    db.refresh(check)
    return check


# ============================================================================
# 4. 订购
# ============================================================================

def _gen_order_no(db: Session) -> str:
    """生成订购单号：日期(YYYYMMDD) + 当日顺序号(2 位)，如 2026072701。"""
    today = date.today()
    prefix = today.strftime("%Y%m%d")
    cnt = db.query(ReagentOrder).filter(ReagentOrder.order_no.like(f"{prefix}%")).count()
    seq = cnt + 1
    candidate = f"{prefix}{seq:02d}"
    # 极端并发兜底：若碰撞则顺延
    while db.query(ReagentOrder).filter(ReagentOrder.order_no == candidate).first():
        seq += 1
        candidate = f"{prefix}{seq:02d}"
    return candidate


@router.get("/orders", response_model=dict)
def list_orders(
    library: Optional[str] = Query(None, description="责任库筛选（生化凝血/免疫）"),
    page: int = Query(1, ge=1),
    page_size: int = Query(20, ge=1, le=100),
    db: Session = Depends(get_db),
    user: User = Depends(get_current_user),
):
    base = db.query(ReagentOrder)
    if library:
        base = base.filter(ReagentOrder.library == library)
    # 试剂配送：仅看自己建的
    roles = set((user.roles or "").split(",")) | {user.role}
```

```
    if "reagent_delivery" in roles and "admin" not in roles and "reagent_manager" not in roles:
        base = base.filter(ReagentOrder.created_by == user.username)
    total = base.count()
    rows = base.order_by(ReagentOrder.order_date.desc()).offset(
        (page - 1) * page_size
    ).limit(page_size).all()
    return {"total": total, "page": page, "page_size": page_size,
            "items": [ReagentOrderRead.model_validate(r) for r in rows]}


@router.get("/orders/_next-no")
def next_order_no(db: Session = Depends(get_db), _=Depends(get_current_user)):
    """返回建议的下一个订购单号（日期+顺序）。"""
    return {"order_no": _gen_order_no(db)}


@router.get("/orders/{order_id}", response_model=ReagentOrderRead)
def get_order(order_id: int, db: Session = Depends(get_db), _=Depends(get_current_user)):
    order = db.query(ReagentOrder).get(order_id)
    if not order:
        raise HTTPException(404, "订购单未找到")
    return order


@router.post("/orders", response_model=ReagentOrderRead)
def create_order(
    data: ReagentOrderCreate, db: Session = Depends(get_db),
    user: User = Depends(require_roles("admin", "reagent_manager")),
):
    order_no = data.order_no or _gen_order_no(db)
    # 唯一性兜底（极端并发下若用户提供或生成值已存在则重生成）
    while db.query(ReagentOrder).filter(ReagentOrder.order_no == order_no).first():
        order_no = _gen_order_no(db)
    order = ReagentOrder(
        library=data.library or "", order_no=order_no, order_date=data.order_date,
        order_type=data.order_type, status="草稿",
        operator=user.full_name or user.username, remark=data.remark,
        created_by=user.username,
    )
    db.add(order)
    db.flush()
    for it in data.items:
        db.add(ReagentOrderItem(order_id=order.id, **it.model_dump()))
    db.commit()
    db.refresh(order)
    return order


@router.put("/orders/{order_id}", response_model=ReagentOrderRead)
def update_order(
```

```
    order_id: int, data: ReagentOrderCreate, db: Session = Depends(get_db),
    user: User = Depends(require_roles("admin", "reagent_manager")),
):
    order = db.query(ReagentOrder).get(order_id)
    if not order:
        raise HTTPException(404, "订购单未找到")
    # 已确认提交不可改（防库存脱节）
    if order.is_confirmed:
        raise HTTPException(403, "该订购单已确认提交，不可再修改")
    # 试剂配送仅能改自己建的
    roles = set((user.roles or "").split(",")) | {user.role}
    if "reagent_delivery" in roles and "admin" not in roles and "reagent_manager" not in roles:
        if order.created_by and order.created_by != user.username:
            raise HTTPException(403, "只能修改自己创建的订购单")
    order.library = data.library or ""
    order.order_date = data.order_date
    order.order_type = data.order_type
    order.status = data.status
    order.remark = data.remark
    order.operator = user.full_name or user.username
    # 先删旧细项再重建
    db.query(ReagentOrderItem).filter(ReagentOrderItem.order_id == order.id).delete()
    for it in data.items:
        db.add(ReagentOrderItem(order_id=order.id, **it.model_dump()))
    db.commit()
    db.refresh(order)
    return order


@router.post("/orders/{order_id}/confirm", response_model=ReagentOrderRead)
def confirm_order(
    order_id: int, db: Session = Depends(get_db),
    user: User = Depends(require_roles("admin", "reagent_manager")),
):
    """确认提交订购单：锁定不可再改（不影响库存，仅作状态/权限控制）。"""
    order = db.query(ReagentOrder).get(order_id)
    if not order:
        raise HTTPException(404, "订购单未找到")
    if order.is_confirmed:
        raise HTTPException(400, "该订购单已确认提交")
    order.is_confirmed = True
    order.confirmed_at = datetime.utcnow()
    order.confirmed_by = user.full_name or user.username
    order.status = "已提交"
    db.commit()
    db.refresh(order)
    return order


@router.get("/orders/{order_id}/export-form")
```

```
def export_order_form(
    order_id: int, db: Session = Depends(get_db),
    _=Depends(get_current_user),
):
    """导出向设备科提交的订购表（含材料编码/名称/规格/品牌/数量/单位）。"""
    order = db.query(ReagentOrder).get(order_id)
    if not order:
        raise HTTPException(404, "订购单未找到")
    try:
        import openpyxl
        from openpyxl.styles import Font, Alignment, Border, Side
    except ImportError:
        raise HTTPException(400, "服务端未安装 openpyxl，无法导出 Excel")

    wb = openpyxl.Workbook()
    ws = wb.active
    ws.title = "设备科订购表"
    headers = ["材料编码", "试剂名称", "规格", "品牌", "订购数量", "单位", "备注"]
    ws.append(headers)
    head_font = Font(bold=True, color="FFFFFF")
    head_fill = openpyxl.styles.PatternFill("solid", fgColor="1A365D")
    thin = Side(style="thin", color="D0D5DD")
    border = Border(left=thin, right=thin, top=thin, bottom=thin)
    for c in range(1, len(headers) + 1):
        cell = ws.cell(row=1, column=c)
        cell.font = head_font
        cell.fill = head_fill
        cell.alignment = Alignment(horizontal="center", vertical="center")
        cell.border = border
    for it in order.items:
        item = db.query(ReagentItem).get(it.item_id)
        ws.append([
            item.material_code if item else "",
            item.name if item else f"(id={it.item_id})",
            item.spec if item else "",
            item.brand if item else "",
            it.ordered_quantity,
            item.unit if item else "",
            it.remark or "",
        ])
    for r in range(2, ws.max_row + 1):
        for c in range(1, len(headers) + 1):
            ws.cell(row=r, column=c).border = border
    widths = [16, 28, 22, 14, 12, 8, 24]
    for i, w in enumerate(widths, 1):
        ws.column_dimensions[openpyxl.utils.get_column_letter(i)].width = w

    buf = BytesIO()
    wb.save(buf)
    buf.seek(0)
```

```
    filename = f"设备科订购表_{order.order_no}.xlsx"
    return StreamingResponse(
        buf,
        media_type="application/vnd.openxmlformats-officedocument.spreadsheetml.sheet",
        headers={"Content-Disposition": f'attachment; filename="{filename}"'},
    )


# ============================================================================
# 5. 到货接收
# ============================================================================

@router.get("/receivings", response_model=dict)
def list_receivings(
    library: Optional[str] = Query(None, description="责任库筛选（生化凝血/免疫），按收货明细试剂归属过滤"),
    confirmed: Optional[bool] = Query(None, description="筛选确认状态：true=已确认，false=待确认"),
    page: int = Query(1, ge=1),
    page_size: int = Query(20, ge=1, le=100),
    db: Session = Depends(get_db),
    user: User = Depends(get_current_user),
):
    base = db.query(Receiving)
    if library:
        lib_item_ids = [r[0] for r in db.query(ReagentItem.id).filter(ReagentItem.library == library).all()]
        recv_ids = [r[0] for r in db.query(ReceivingItem.receiving_id).filter(
            ReceivingItem.item_id.in_(lib_item_ids)).distinct().all()]
        base = base.filter(Receiving.id.in_(recv_ids))
    if confirmed is not None:
        base = base.filter(Receiving.is_confirmed == confirmed)
    # 试剂配送角色：仅能看自己创建的记录
    owned = set(user_roles_list(user))
    is_delivery_only = ("reagent_delivery" in owned) and ("admin" not in owned) and ("reagent_manager" not in owned)
    if is_delivery_only:
        base = base.filter(Receiving.created_by == user.username)
    total = base.count()
    rows = base.order_by(Receiving.receipt_date.desc()).offset(
        (page - 1) * page_size
    ).limit(page_size).all()
    return {"total": total, "page": page, "page_size": page_size,
            "items": [ReceivingRead.model_validate(r) for r in rows]}


@router.get("/receivings/{receiving_id}", response_model=ReceivingRead)
def get_receiving(receiving_id: int, db: Session = Depends(get_db), _=Depends(get_current_user)):
    r = db.query(Receiving).get(receiving_id)
    if not r:
        raise HTTPException(404, "收货记录未找到")
    return r
```

```
@router.post("/receivings", response_model=ReceivingRead)
def create_receiving(
    data: ReceivingCreate, db: Session = Depends(get_db),
    user: User = Depends(require_roles("admin", "reagent_manager", "reagent_delivery")),
):
    rec = Receiving(
        receipt_no=data.receipt_no, receipt_date=data.receipt_date,
        order_id=data.order_id, delivery_person=data.delivery_person,
        receiver=data.receiver or "",  # 新建/编辑时保存用户实际填入值（可空，由确认时填入确认人）
        remark=data.remark, created_by=user.username, is_confirmed=False,
    )
    db.add(rec)
    db.flush()
    for it in data.items:
        db.add(ReceivingItem(
            receiving_id=rec.id, item_id=it.item_id, batch_no=it.batch_no,
            expiry_date=it.expiry_date, quantity=it.quantity, remark=it.remark,
        ))
    # 注意：新建时**不**直接入库存，待「确认接收」后才写入实时库存（避免未确认单据污染库存）
    db.commit()
    db.refresh(rec)
    return rec


@router.post("/receivings/{receiving_id}/confirm", response_model=ReceivingRead)
def confirm_receiving(
    receiving_id: int, db: Session = Depends(get_db),
    user: User = Depends(require_roles("admin", "reagent_manager")),
):
    """确认接收：写入实时库存，并记录确认人/确认时间。重复确认不会重复加库存。"""
    r = db.query(Receiving).get(receiving_id)
    if not r:
        raise HTTPException(404, "收货记录未找到")
    if r.is_confirmed:
        raise HTTPException(400, "该收货单已确认接收，无需重复确认")
    # 接收人在「确认接收」时强制填入当前登录人（即便保存时已有人工填值）
    r.receiver = user.full_name or user.username
    for it in r.items:
        stock = db.query(ReagentStock).filter(
            ReagentStock.item_id == it.item_id,
            ReagentStock.batch_no == it.batch_no,
        ).first()
        if stock:
            stock.quantity += it.quantity
            stock.last_updated = datetime.utcnow()
        else:
            db.add(ReagentStock(
                item_id=it.item_id, batch_no=it.batch_no,
                expiry_date=it.expiry_date, quantity=it.quantity,
            ))
```

```
    r.is_confirmed = True
    r.confirmed_at = datetime.utcnow()
    r.confirmed_by = user.username
    db.commit()
    db.refresh(r)
    return r


@router.delete("/receivings/{receiving_id}")
def delete_receiving(
    receiving_id: int, db: Session = Depends(get_db),
    user: User = Depends(require_roles("admin")),
):
    """管理员删除收货单（含库存回退）。仅 admin 可调用，用于清理测试数据/误操作。"""
    r = db.query(Receiving).get(receiving_id)
    if not r:
        raise HTTPException(404, "收货记录未找到")
    # 若已确认，回退库存
    if r.is_confirmed:
        for it in r.items:
            stock = db.query(ReagentStock).filter(
                ReagentStock.item_id == it.item_id,
                ReagentStock.batch_no == (it.batch_no or ""),
            ).first()
            if stock:
                stock.quantity -= it.quantity
                if stock.quantity <= 0:
                    db.delete(stock)
                else:
                    stock.last_updated = datetime.utcnow()
    # 删除收货细项
    for it in r.items:
        db.delete(it)
    # 删除收货主记录
    db.delete(r)
    db.commit()
    return {"ok": True, "message": f"收货单 {r.receipt_no} 已删除，库存已回退"}


@router.put("/receivings/{receiving_id}", response_model=ReceivingRead)
def update_receiving(
    receiving_id: int, data: ReceivingCreate, db: Session = Depends(get_db),
    user: User = Depends(require_roles("admin", "reagent_manager", "reagent_delivery")),
):
    """编辑收货单。规则：
    - 已确认接收的单据不可再修改（避免与已入库库存脱节）。
    - 试剂配送角色只能修改自己创建的单据（created_by == 当前用户）。
    """
    r = db.query(Receiving).get(receiving_id)
    if not r:
```

```
        raise HTTPException(404, "收货记录未找到")
    owned = set(user_roles_list(user))
    is_restricted = ("reagent_delivery" in owned) and ("admin" not in owned) and ("reagent_manager" not in owned)
    if is_restricted and r.created_by != user.username:
        raise HTTPException(403, "只能修改自己创建的收货记录")
    if r.is_confirmed:
        raise HTTPException(400, "已确认接收，不能修改")
    r.receipt_date = data.receipt_date
    r.order_id = data.order_id
    r.delivery_person = data.delivery_person
    r.receiver = data.receiver or r.receiver
    r.remark = data.remark
    # 替换明细
    for it in r.items:
        db.delete(it)
    db.flush()
    for it in data.items:
        db.add(ReceivingItem(
            receiving_id=r.id, item_id=it.item_id, batch_no=it.batch_no,
            expiry_date=it.expiry_date, quantity=it.quantity, remark=it.remark,
        ))
    db.commit()
    db.refresh(r)
    return r


# ============================================================================
# 6. 月消耗
# ============================================================================

@router.get("/consumption", response_model=dict)
def list_consumption(
    year_month: Optional[str] = Query(None, description="YYYY-MM"),
    library: Optional[str] = Query(None, description="责任库筛选（生化凝血/免疫）"),
    q: str = Query("", description="搜索试剂名称/品牌"),
    page: int = Query(1, ge=1),
    page_size: int = Query(50, ge=1, le=200),
    db: Session = Depends(get_db),
    _=Depends(get_current_user),
):
    base = db.query(ReagentConsumption)
    if year_month:
        base = base.filter(ReagentConsumption.year_month == year_month)
    if library or q.strip():
        ri_base = db.query(ReagentItem.id)
        if library:
            ri_base = ri_base.filter(ReagentItem.library == library)
        if q.strip():
            kw = f"%{q.strip()}%"
            ri_base = ri_base.filter(or_(ReagentItem.name.like(kw), ReagentItem.brand.like(kw)))
```

```
        item_ids = [r[0] for r in ri_base.all()]
        base = base.filter(ReagentConsumption.item_id.in_(item_ids))
    total = base.count()
    rows = base.order_by(ReagentConsumption.year_month.desc(), ReagentConsumption.item_id).offset(
        (page - 1) * page_size
    ).limit(page_size).all()
    return {"total": total, "page": page, "page_size": page_size,
            "items": [ReagentConsumptionRead.model_validate(r) for r in rows]}


@router.post("/consumption/_calculate")
def calculate_consumption(
    year_month: str = Query(..., description="YYYY-MM"),
    db: Session = Depends(get_db),
    user: User = Depends(require_roles("admin", "reagent_manager", "lab_technician")),
):
    """盘库后触发：对所有 reagent_items 计算月消耗。

    公式：月消耗 = 期初库存(opening) + 本月入库(received) - 期末库存(closing)
      - 期末库存 = 当前实时库存合计（月末盘库已回写 ReagentStock）
      - 本月入库 = 当月到货接收合计（Receiving.receipt_date 在区间内）
      - 期初库存 = 上月消耗记录的期末库存
    """
    if not (len(year_month) == 7 and year_month[4] == "-"):
        raise HTTPException(400, "year_month 格式应为 YYYY-MM")
    try:
        y, m = int(year_month[:4]), int(year_month[5:7])
    except ValueError:
        raise HTTPException(400, "year_month 格式应为 YYYY-MM")
    month_start = date(y, m, 1)
    if m == 12:
        month_end = date(y + 1, 1, 1)
    else:
        month_end = date(y, m + 1, 1)

    items = db.query(ReagentItem).filter(ReagentItem.is_active == True).all()
    added, updated = 0, 0
    for it in items:
        # 期末库存：当前实时库存合计
        closing_qty = db.query(func.coalesce(func.sum(ReagentStock.quantity), 0)).filter(
            ReagentStock.item_id == it.id
        ).scalar() or 0
        # 本月入库：当月到货接收细项合计
        received_qty = db.query(func.coalesce(func.sum(ReceivingItem.quantity), 0)).join(
            Receiving, ReceivingItem.receiving_id == Receiving.id
        ).filter(
            ReceivingItem.item_id == it.id,
            Receiving.receipt_date >= month_start,
            Receiving.receipt_date < month_end,
        ).scalar() or 0
```

```
        # 期初库存：上月消耗记录的期末库存
        prev = db.query(ReagentConsumption).filter(
            ReagentConsumption.item_id == it.id,
        ).order_by(ReagentConsumption.year_month.desc()).first()
        opening = prev.closing_balance if prev else 0
        consumption = opening + received_qty - closing_qty
        if consumption < 0:
            consumption = 0

        existing = db.query(ReagentConsumption).filter(
            ReagentConsumption.item_id == it.id,
            ReagentConsumption.year_month == year_month,
        ).first()
        if existing:
            existing.opening_balance = opening
            existing.total_received = received_qty
            existing.closing_balance = closing_qty
            existing.consumption = consumption
            existing.calculated_at = datetime.utcnow()
            updated += 1
        elif opening or received_qty or closing_qty:
            # 仅当有数据时落库，避免大量空行
            db.add(ReagentConsumption(
                item_id=it.id, year_month=year_month,
                opening_balance=opening, total_received=received_qty,
                closing_balance=closing_qty, consumption=consumption,
            ))
            added += 1
    db.commit()
    return {"added": added, "updated": updated, "year_month": year_month}


# ============================================================================
# 7. Excel 导入试剂目录
# ============================================================================

@router.post("/items/_import-excel", response_model=ImportResult)
def import_reagent_from_excel(
    db: Session = Depends(get_db),
    user: User = Depends(require_roles("admin")),
    file: UploadFile = File(...),
):
    """从 Excel 导入试剂目录（要求：第 1 行表头，需含 'name' 或 '试剂名称' 列）。"""
    result = ImportResult()
    try:
        import openpyxl
    except ImportError:
        raise HTTPException(400, "服务端未安装 openpyxl，无法处理 Excel")
    wb = openpyxl.load_workbook(file.file)
    ws = wb.active
```

```
    rows = list(ws.iter_rows(values_only=True))
    if not rows:
        raise HTTPException(400, "Excel 为空")
    headers = [str(h or "").strip() for h in rows[0]]
    # 尝试推断列映射
    col_map = {}
    for col in ["name", "试剂名称", "材料名称", "名称", "名称(试剂名称)", "试剂名"]:
        if col in headers:
            col_map["name"] = headers.index(col)
            break
    if "name" not in col_map:
        raise HTTPException(400, "找不到 'name/试剂名称' 列")
    for col in ["type", "类型", "试剂类型"]:
        if col in headers:
            col_map["type"] = headers.index(col)
            break
    for col in ["category", "类别", "专业", "专业组"]:
        if col in headers:
            col_map["category"] = headers.index(col)
            break
    for col in ["library", "责任库", "库"]:
        if col in headers:
            col_map["library"] = headers.index(col)
            break
    for col in ["brand", "品牌", "生产厂家", "厂家"]:
        if col in headers:
            col_map["brand"] = headers.index(col)
            break
    for col in ["spec", "规格", "规格型号", "包装规格"]:
        if col in headers:
            col_map["spec"] = headers.index(col)
            break
    for col in ["material_code", "材料编码", "编码", "订购编码"]:
        if col in headers:
            col_map["material_code"] = headers.index(col)
            break
    for col in ["unit", "单位"]:
        if col in headers:
            col_map["unit"] = headers.index(col)
            break
    for col in ["manufacturer", "生产厂家 2", "制造商"]:
        if col in headers:
            col_map["manufacturer"] = headers.index(col)
            break
    for col in ["supplier", "供应商"]:
        if col in headers:
            col_map["supplier"] = headers.index(col)
            break
    for col in ["unit_price", "单价", "价格", "参考单价"]:
        if col in headers:
```

```
            col_map["unit_price"] = headers.index(col)
            break
    for col in ["remark", "备注", "说明"]:
        if col in headers:
            col_map["remark"] = headers.index(col)
            break

    imported = 0
    skipped = 0
    for i, row in enumerate(rows[1:], 2):
        try:
            name = str(row[col_map["name"]] or "").strip()
            if not name:
                skipped += 1
                continue
            # 主去重键：材料编码优先，回退名称
            code = str(row[col_map["material_code"]]).strip() if "material_code" in col_map else ""
            existing = None
            if code:
                existing = db.query(ReagentItem).filter(ReagentItem.material_code == code).first()
            if existing is None:
                existing = db.query(ReagentItem).filter(ReagentItem.name == name).first()
            if existing:
                # 已存在：用本次导入信息补全/覆盖目录字段
                for key in ("material_code", "spec", "brand", "unit", "category", "library",
                            "type", "unit_price", "manufacturer", "supplier", "remark"):
                    if key not in col_map:
                        continue
                    v = row[col_map[key]]
                    if v is None or str(v).strip() == "":
                        continue
                    if key == "unit_price":
                        try:
                            setattr(existing, key, Decimal(str(v)))
                        except Exception:
                            pass
                    elif key == "category":
                        val = str(v).strip()
                        setattr(existing, key, val)
                        if "library" not in col_map:
                            setattr(existing, "library", derive_library(val))
                    elif key == "library":
                        setattr(existing, key, str(v).strip())
                    elif key == "type":
                        setattr(existing, key, str(v).strip())
                    else:
                        setattr(existing, key, str(v).strip())
                skipped += 1
                continue
            item = ReagentItem(name=name)
```

```
            if "type" in col_map:
                item.type = str(row[col_map["type"]] or "试剂").strip()
            else:
                item.type = detect_reagent_type(name)  # 按名称自动识别
            if "category" in col_map:
                item.category = str(row[col_map["category"]] or "").strip()
            if "library" in col_map:
                item.library = str(row[col_map["library"]] or "").strip()
            elif item.category:
                item.library = derive_library(item.category)  # 由专业组推导责任库
            if "brand" in col_map:
                item.brand = str(row[col_map["brand"]] or "").strip()
            if "spec" in col_map:
                item.spec = str(row[col_map["spec"]] or "").strip()
            if "material_code" in col_map:
                item.material_code = code
            if "unit" in col_map:
                item.unit = str(row[col_map["unit"]] or "").strip()
            if "manufacturer" in col_map:
                item.manufacturer = str(row[col_map["manufacturer"]] or "").strip()
            if "supplier" in col_map:
                item.supplier = str(row[col_map["supplier"]] or "").strip()
            if "remark" in col_map:
                item.remark = str(row[col_map["remark"]] or "").strip()
            if "unit_price" in col_map:
                v = row[col_map["unit_price"]]
                if v is not None and str(v).strip() != "":
                    try:
                        item.unit_price = Decimal(str(v))
                    except Exception:
                        pass
            db.add(item)
            imported += 1
        except Exception as e:
            result.errors.append(f"第{i}行: {e}")
            skipped += 1
    db.commit()
    result.total = len(rows) - 1
    result.imported = imported
    result.skipped = skipped
    return result


# ============================================================================
# 8. 项目 ↔ 试剂 / 仪器 ↔ 耗材 关联管理
# ============================================================================

import re as _re
```

```
def _norm_match(s: str) -> str:
    """归一化用于模糊匹配：仅去空白/标点与 R1/R2 标记，保留字母数字与中文
    （刻意保留型号代码 DxI800 / ACL / cobas 等，以便耗材能按仪器型号匹配）。"""
    s = str(s or "")
    s = _re.sub(r"[Rr][1-9]\b", "", s)                # 去 R1/R2 标记（避免干扰）
    s = _re.sub(r"[^0-9A-Za-z\u4e00-\u9fff]", "", s)  # 仅留字母数字与中文（保留型号代码）
    return s.strip().lower()


# 过于通用的「抗体类别代号」：单独出现时无法区分具体项目，若作为匹配键会让
# 「免疫球蛋白 G」被所有含 "igg" 的试剂盒（单纯疱疹/肝炎/风疹…）误命中。
# 仅当试剂名包含项目的「具体全称」（如「免疫球蛋白 g」）时才允许匹配。
GENERIC_CLASS_TOKENS = {"igg", "igm", "iga", "ige", "ig"}


def _alnum_tokens(s: str) -> list:
    """从仪器名/型号或耗材名中提取「纯 ASCII 型号代码」片段。

    直接按正则提取所有 ASCII 字母/数字连续段（忽略中文与任何括号/标点），
    只保留「长度 >= 2 且至少含一个字母」的片段（如 au5822 / dxi800c /
    top700a / e601 / acl / top / au / bm60）。

    - 要求含字母：排除纯数字片段（300 / 600 / 618 …），否则仪器型号里的数字
      会误命中耗材名中的货号（如 03005712190 含 300），造成错误关联。
    - 品牌中文前缀（贝克曼/罗氏…）天然被排除；即便型号被全角括号包裹
      （如「反应杯（DxI800）」）也能正确析出 `dxi800`。
    """
    return [t.lower() for t in _re.findall(r"[A-Za-z0-9]+", str(s or ""))
            if len(t) >= 2 and any(c.isalpha() for c in t)]


def _core_name(s: str) -> str:
    """提取检验项目「核心词」：去掉括号内容（如 (ATIII)）与末尾罗马数字，
    便于试剂名按核心词匹配—「抗凝血酶 III」→ 抗凝血酶，能命中
    「抗凝血酶测定试剂盒（发色底物法）」这类本可关联却因带后缀而漏匹配的试剂。"""
    s = _re.sub(r"\([^)]*\)", "", str(s or ""))
    s = _re.sub(r"[IVX]+$", "", s)
    return _norm_match(s)


# 试剂名尾部通用后缀—提取「试剂核心词」时逐级剥离，露出真正的测定对象
_REAGENT_SUFFIXES = ["测定试剂盒", "检测试剂盒", "试剂盒", "校准品", "质控品",
                     "质控血清", "测定", "检测", "试剂"]


def _reagent_core(s: str) -> str:
    """提取试剂「核心词」：去方法括号 + 逐级剥离尾部通用后缀（测定试剂盒/校准品…）。
    例：「D-二聚体测定试剂盒（免疫比浊法）」→ d 二聚体，可命中项目「血浆 D-二聚体」；
    「铁测定试剂盒」→ 铁，可命中项目「铁」。解决项目名带前缀/后缀导致漏匹配的问题。"""
    s = _re.sub(r"\([^)]*\)", "", str(s or ""))
```

```
    s = _re.sub(r"（[^）]*）", "", s)
    changed = True
    while changed:
        changed = False
        for suf in _REAGENT_SUFFIXES:
            if s.endswith(suf) and len(s) > len(suf):
                s = s[: -len(suf)]
                changed = True
                break
    return _norm_match(s)


def _best_test_item_match(rn: str, rc: str, test_norms):
    """在 test_norms[(id, tn, tn_core, aliases)] 中为试剂找最佳匹配项目。
    rn=试剂名归一化，rc=试剂核心词。双向包含均计分；试剂核心与项目名完全相等（如 铁/锌）给通过阈值。
    返回 (best_test_item_id, best_score)。"""
    best, best_score = None, 0
    for tid, tn, tn_core, tals in test_norms:
        for cand in ([tn, tn_core] + tals):
            if not cand or cand in GENERIC_CLASS_TOKENS:
                continue
            score = 0
            if cand in rn or cand in rc:
                score = len(cand)
            elif rn in cand or (rc and rc in cand):
                score = len(rc) if (rc and rc in cand) else len(rn)
            if rc and cand == rc:
                score = max(score, 2)
            if score > best_score:
                best_score, best = score, tid
    return best, best_score


@router.post("/associations/_auto-match", response_model=dict)
def auto_match_associations(
    reset: bool = Query(False, description="true 时先清空已有自动匹配记录再重新生成"),
    db: Session = Depends(get_db),
    user: User = Depends(require_roles("admin", "reagent_manager")),
):
    """根据名称模糊匹配，自动生成 项目↔试剂 与 耗材↔仪器 关联。

    - 试剂/校准品/质控品：名称 vs 项目 name/aliases 模糊匹配，role 取试剂类型(试剂/校准品/质控品)。
    - 耗材：名称 vs 仪器 name/model 关键词匹配，role=耗材。
    返回新增条数与无法自动匹配（需人工审核）的清单。
    """
    if reset:
        db.query(TestItemReagent).delete()
        db.query(InstrumentReagent).delete()

    tests = db.query(TestItem).all()
```

```
    test_norms = [
        (t.id, _norm_match(t.name), _core_name(t.name),
         [_norm_match(a) for a in str(t.aliases or "").split(",") if a.strip()])
        for t in tests
    ]
    reagents = db.query(ReagentItem).all()

    tir_added, tir_unmatched = 0, []
    for r in reagents:
        if r.type == "耗材":
            continue
        rn = _norm_match(r.name)
        rc = _reagent_core(r.name)
        best, best_score = _best_test_item_match(rn, rc, test_norms)
        if best and best_score >= 2:
            if not db.query(TestItemReagent).filter_by(test_item_id=best, reagent_item_id=r.id).first():
                db.add(TestItemReagent(
                    test_item_id=best, reagent_item_id=r.id,
                    role=r.type if r.type in ("校准品", "质控品") else "试剂",
                    auto_matched=True,
                ))
                tir_added += 1
        else:
            tir_unmatched.append({"reagent_id": r.id, "name": r.name, "type": r.type})

    insts = db.query(Instrument).all()
    # 预计算每个仪器的：全名归一化 / ASCII 型号代码列表 / 中文名归一化
    inst_idx = []
    for ins in insts:
        full = _norm_match((ins.name or "") + " " + (getattr(ins, "model", None) or ""))
        alnum = _alnum_tokens((ins.name or "") + " " + (getattr(ins, "model", None) or ""))
        cn_name = _norm_match(ins.name or "")
        inst_idx.append((ins.id, full, alnum, cn_name))

    ir_added, ir_unmatched = 0, []
    for r in reagents:
        if r.type != "耗材":
            continue
        rn = _norm_match(r.name)
        r_alnum = _alnum_tokens(r.name)
        matched_iids = []
        for iid, full, alnum, cn_name in inst_idx:
            score = 0
            # 1) 仪器型号代码出现在耗材名中（强信号：耗材明确写了该型号）
            for t in alnum:
                if len(t) >= 3 and t in rn:
                    score = max(score, len(t))
            # 2) 耗材型号代码出现在仪器全名中（耗材带的型号代码命中仪器）
            for t in r_alnum:
                if len(t) >= 2 and t in full:
```

```
                    score = max(score, len(t))
            # 3) 仪器中文名称出现在耗材名中（如「糖化血红蛋白分析仪」）
            if len(cn_name) >= 4 and cn_name in rn:
                score = max(score, len(cn_name))
            if score >= 3:
                matched_iids.append(iid)
        if matched_iids:
            for iid in matched_iids:
                if not db.query(InstrumentReagent).filter_by(instrument_id=iid, reagent_item_id=r.id).first():
                    db.add(InstrumentReagent(instrument_id=iid, reagent_item_id=r.id, role="耗材", auto_matched=True))
                    ir_added += 1
        else:
            ir_unmatched.append({"reagent_id": r.id, "name": r.name})

    db.commit()
    return {
        "test_item_reagents_added": tir_added,
        "instrument_reagents_added": ir_added,
        "test_item_unmatched": tir_unmatched,
        "instrument_unmatched": ir_unmatched,
    }


def _enrich_test_item_reagents(rows):
    out = []
    for r in rows:
        ti = r.test_item
        ri = r.reagent_item
        out.append({
            "id": r.id, "test_item_id": r.test_item_id, "reagent_item_id": r.reagent_item_id,
            "role": r.role, "auto_matched": r.auto_matched, "remark": r.remark,
            "test_item_name": ti.name if ti else "",
            "reagent_name": ri.name if ri else "", "reagent_type": ri.type if ri else "",
            "reagent_library": ri.library if ri else "",
        })
    return out


# 仪器家族映射表（用户 2026-07-26 确认）。
# 键为仪器 name（精确匹配）；值为目标总型号，None 表示无耗材不显示。
# 特殊：name 相同但 model 不同的（如"免疫分析仪"有 e601/e602/e411/A6200），
#       在 _extract_instrument_base_model 中用 _IMMUNO_MODEL_MAP 按 model 二次区分。
INSTRUMENT_FAMILY_MAP = {
    "AU5800": "AU58",              # AU5800 属于 AU58 家族
    "TOP700A": "TOP700",           # TOP700 系列（A/B/C）合并
    "TOP700B": "TOP700",
    "TOP700C": "TOP700",
    "免疫分析仪": "罗氏 Cobas6000",  # 默认归 Cobas6000（e601/e602），e411/A6200 在下面按 model 覆盖
    "DXA5000": None,               # 无耗材，不显示
}
```

```
# 免疫分析仪的 model → 总型号覆盖（name 都是"免疫分析仪"，靠 model 区分）
_IMMUNO_MODEL_MAP = {
    "罗氏 e411": "罗氏 e411",          # 耗材独立，不通用于 Cobas6000
    "安图 A6200": "安图 A6200",        # 耗材独立，不通用于 Cobas6000
}


def _extract_instrument_base_model(name: str, model: str = "") -> str:
    """从仪器 name/model 中提取「总型号」，用于合并同型号多台仪器的耗材关联。

    优先级：① 精确家族映射(name) + 免疫 model 覆盖 → ② 正则提取（去尾部编号）
            → ③ model 提取核心型号 → ④ 兜底返回原 name
    """
    if not name:
        return (model or "").strip()
    raw_name = name.strip()
    raw_model = (model or "").strip()

    # ① 精确家族映射（name 级）
    if raw_name in INSTRUMENT_FAMILY_MAP:
        mapped = INSTRUMENT_FAMILY_MAP[raw_name]
        # 免疫分析仪特殊处理：按 model 再细分
        if raw_name == "免疫分析仪" and raw_model in _IMMUNO_MODEL_MAP:
            return _IMMUNO_MODEL_MAP[raw_model]
        return mapped

    # ② 正则提取总型号（遇空格/中文即停，去尾部实例编号）
    m = re.match(r'^([A-Za-z0-9]+(?:-[A-Za-z0-9]+)*)', raw_name)
    if m:
        base = m.group(1)
        base = re.sub(r'-\d+$', '', base)   # AU58-1 → AU58
        if not base.isdigit() and len(base) >= 2:
            return INSTRUMENT_FAMILY_MAP.get(base, base)
    # ③ 从 model 提取核心型号
    if raw_model:
        m2 = re.search(r'([A-Za-z]{2,}\d*[A-Za-z]*)', raw_model)
        if m2 and len(m2.group(1)) >= 2:
            return INSTRUMENT_FAMILY_MAP.get(m2.group(1), m2.group(1))
    return raw_name


def _enrich_instrument_reagents(rows):
    out = []
    for r in rows:
        ins = r.instrument
        ri = r.reagent_item
        out.append({
            "id": r.id, "instrument_id": r.instrument_id, "reagent_item_id": r.reagent_item_id,
            "role": r.role, "auto_matched": r.auto_matched, "remark": r.remark,
```

```
            "instrument_name": ins.name if ins else "",
            "instrument_model": ins.model if ins else "",
            "reagent_name": ri.name if ri else "", "reagent_type": ri.type if ri else "",
            "reagent_library": ri.library if ri else "",
        })
    return out


@router.get("/associations/suggestions", response_model=dict)
def association_suggestions(
    library: str = Query("", description="责任库过滤（空=全部）"),
    db: Session = Depends(get_db),
    _=Depends(get_current_user),
):
    """找出「本可关联却未关联」的试剂，供人工审核采纳。

    返回四组：
    - test_item_suggestions：未关联任何项目的 试剂/校准品/质控品，按名称对项目模糊匹配得到候选(candidate_test_item_id)，score>=2；
    - test_item_orphans：未关联且无明显候选匹配的上述试剂（需人工判断）；
    - instrument_suggestions：未关联任何仪器的 耗材，按名称对仪器模糊匹配得到候选(instrument_id 列表，含得分)，score>=3；
    - instrument_orphans：未关联且无候选匹配的 耗材。
    停用仪器与停用试剂不参与匹配。
    """
    linked_test_ids = {r.reagent_item_id for r in db.query(TestItemReagent.reagent_item_id).all()}
    linked_inst_ids = {r.reagent_item_id for r in db.query(InstrumentReagent.reagent_item_id).all()}

    tests = db.query(TestItem).all()
    test_norms = [
        (t.id, _norm_match(t.name), _core_name(t.name),
         [_norm_match(a) for a in str(t.aliases or "").split(",") if a.strip()])
        for t in tests
    ]
    insts = db.query(Instrument).all()
    inst_idx = []
    for ins in insts:
        if getattr(ins, "status", None) == "停用":
            continue
        full = _norm_match((ins.name or "") + " " + (getattr(ins, "model", None) or ""))
        alnum = _alnum_tokens((ins.name or "") + " " + (getattr(ins, "model", None) or ""))
        cn_name = _norm_match(ins.name or "")
        inst_idx.append((ins.id, ins.name, getattr(ins, "model", None), full, alnum, cn_name))

    reagents = db.query(ReagentItem).filter(ReagentItem.is_active == True).all()

    test_item_suggestions, test_item_orphans = [], []
    instrument_suggestions, instrument_orphans = [], []
    for r in reagents:
        if library and r.library != library:
            continue
        if r.type == "耗材":
```

```
            if r.id in linked_inst_ids:
                continue
            rn = _norm_match(r.name)
            r_alnum = _alnum_tokens(r.name)
            matched = []
            for iid, iname, imodel, full, alnum, cn_name in inst_idx:
                score = 0
                for t in alnum:
                    if len(t) >= 3 and t in rn:
                        score = max(score, len(t))
                for t in r_alnum:
                    if len(t) >= 2 and t in full:
                        score = max(score, len(t))
                if len(cn_name) >= 4 and cn_name in rn:
                    score = max(score, len(cn_name))
                if score >= 3:
                    matched.append((iid, iname, imodel, score))
            if matched:
                matched.sort(key=lambda x: -x[3])
                instrument_suggestions.append({
                    "reagent_id": r.id, "reagent_name": r.name, "reagent_type": r.type, "library": r.library,
                    "candidates": [
                        {"instrument_id": m[0], "instrument_name": m[1], "instrument_model": m[2], "score": m[3]}
                        for m in matched
                    ],
                })
            else:
                instrument_orphans.append({
                    "reagent_id": r.id, "reagent_name": r.name, "reagent_type": r.type, "library": r.library,
                })
        else:
            if r.id in linked_test_ids:
                continue
            rn = _norm_match(r.name)
            rc = _reagent_core(r.name)
            best, best_score = _best_test_item_match(rn, rc, test_norms)
            ti = next((t for t in tests if t.id == best), None)
            if best and best_score >= 2:
                test_item_suggestions.append({
                    "reagent_id": r.id, "reagent_name": r.name, "reagent_type": r.type, "library": r.library,
                    "candidate_test_item_id": best,
                    "candidate_test_item_name": ti.name if ti else "",
                    "score": best_score,
                })
            else:
                test_item_orphans.append({
                    "reagent_id": r.id, "reagent_name": r.name, "reagent_type": r.type, "library": r.library,
                })

    # 按得分降序，便于优先处理高置信度项
```

```
          <el-button size="small" link type="danger" @click="onDeleteCheck(row)" v-if="canWrite">删除</el-button>
        </template>
      </el-table-column>
    </el-table>
    <el-pagination class="pager" v-model:current-page="page" v-model:page-size="pageSize"
      :total="total" :page-sizes="[20,50]" layout="total, sizes, prev, pager, next"
      @current-change="refresh" @size-change="page=1; refresh()" />

    <!-- 新建盘库弹窗 -->
    <el-dialog v-model="dialogVisible" :title="`新建盘库（${reagentStore.library}）`" width="min(960px, 96vw)" top="3vh">
      <el-form :model="checkForm" label-width="80px" size="small">
        <el-row :gutter="12">
          <el-col :span="8"><el-form-item label="日期"><el-date-picker v-model="checkForm.check_date" type="date" value-
format="YYYY-MM-DD" style="width:100%" /></el-form-item></el-col>
          <el-col :span="8"><el-form-item label="类型"><el-select v-model="checkForm.check_type" style="width:100%">
            <el-option label="月末盘库" value="月末盘库" /><el-option label="月中盘库" value="月中盘库" />
          </el-select></el-form-item></el-col>
          <el-col :span="8"><el-form-item label="备注"><el-input v-model="checkForm.remark" /></el-form-item></el-col>
        </el-row>
      </el-form>
      <div style="margin-bottom:8px;display:flex;gap:8px;align-items:center;flex-wrap:wrap">
        <el-select v-model="categoryFilter" placeholder="全部分类" clearable style="width:130px" size="default">
          <el-option label="按品牌" value="brand" />
          <el-option label="试剂 / 校准品" value="reagent" />
          <el-option label="耗材" value="consumable" />
          <el-option label="质控品" value="control" />
        </el-select>
        <el-select v-if="categoryFilter === 'brand'" v-model="brandTypeFilter" placeholder="类型" clearable
style="width:110px" size="default">
          <el-option label="试剂" value="reagent" />
          <el-option label="耗材" value="consumable" />
          <el-option label="质控品" value="control" />
        </el-select>
        <el-input v-model="dialogSearch" placeholder="模糊检索试剂/规格/编码/品牌/项目名(含英文别名如 ALT)..." clearable
style="width:340px" />
        <el-button @click="onPrintBlank" :icon="Printer" size="small">打印空白录入页</el-button>
        <span class="muted" style="font-size:12px">共 {{ totalEntries }} 项</span>
      </div>

      <div class="entry-scroll">
        <!-- 按项目 -->
        <template v-for="grp in filteredProjects" :key="'p'+grp.test_item_id">
          <h4 class="grp-title">项目：{{ grp.test_item_name }}</h4>
          <el-table :data="grp.items" border size="small">
            <el-table-column label="试剂 / 校准品" min-width="190">
              <template #default="{ row }">{{ row.name }} <span
class="muted">{{ row.spec }} · {{ row.brand }}</span></template>
            </el-table-column>
            <el-table-column label="盘点余量" width="120" fixed="right">
              <template #default="{ row }">
                <el-input-number v-model="quantities[row.item_id]" :min="0" size="small" style="width:110px" controls-
position="right" />
              </template>
            </el-table-column>
          </el-table>
```

```
        </template>
        <!-- 按仪器 -->
        <template v-for="grp in filteredInstruments" :key="'i'+grp.group">
          <h4 class="grp-title">仪器：{{ grp.group }} <span class="muted" v-if="grp.instruments.length">
（{{ grp.instruments.join('、') }}）</span></h4>
          <el-table :data="grp.items" border size="small">
            <el-table-column label="耗材" min-width="190">
              <template #default="{ row }">{{ row.name }} <span
class="muted">{{ row.spec }} · {{ row.brand }}</span></template>
            </el-table-column>
            <el-table-column label="盘点余量" width="120" fixed="right">
              <template #default="{ row }">
                <el-input-number v-model="quantities[row.item_id]" :min="0" size="small" style="width:110px" controls-
position="right" />
              </template>
            </el-table-column>
          </el-table>
        </template>
        <!-- 质控品 -->
        <template v-if="filteredControls.length">
          <h4 class="grp-title">质控品（单独）</h4>
          <el-table :data="filteredControls" border size="small">
            <el-table-column label="质控品" min-width="190">
              <template #default="{ row }">{{ row.name }} <span
class="muted">{{ row.spec }} · {{ row.brand }}</span></template>
            </el-table-column>
            <el-table-column label="盘点余量" width="120" fixed="right">
              <template #default="{ row }">
                <el-input-number v-model="quantities[row.item_id]" :min="0" size="small" style="width:110px" controls-
position="right" />
              </template>
            </el-table-column>
          </el-table>
        </template>
        <!-- 按品牌 -->
        <template v-if="filteredByBrand.length">
          <template v-for="grp in filteredByBrand" :key="'b'+grp.brand">
            <h4 class="grp-title">品牌：{{ grp.brand }}</h4>
            <el-table :data="grp.items" border size="small">
              <el-table-column label="名称 / 规格 / 品牌" min-width="200">
                <template #default="{ row }">{{ row.name }} <span
class="muted">{{ row.spec }} · {{ row.brand }}</span></template>
              </el-table-column>
              <el-table-column label="盘点余量" width="120" fixed="right">
                <template #default="{ row }">
                  <el-input-number v-model="quantities[row.item_id]" :min="0" size="small" style="width:110px" controls-
position="right" />
                </template>
              </el-table-column>
            </el-table>
          </template>
        </template>
        <el-empty v-if="totalEntries === 0" description="该分类/检索条件下无数据" />
      </div>

      <template #footer>
        <el-button @click="onPrintBlank" :icon="Printer">打印空白页</el-button>
```

```
        <el-button @click="dialogVisible=false">取消</el-button>
        <el-button type="primary" :loading="submitting" @click="onSubmit">提交盘库</el-button>
      </template>
    </el-dialog>

    <!-- 查看盘库详情 -->
    <el-dialog v-model="viewVisible" :title="`盘库详情（${viewLibrary}）`" width="640px">
      <el-table :data="viewItems" border size="small">
        <el-table-column label="试剂/耗材" min-width="220">
          <template #default="{ row }">{{ row._name || `(id=${row.item_id})` }} <span
class="muted">{{ row._spec }}</span></template>
        </el-table-column>
        <el-table-column label="批号" width="120"><template #default="{ row }">{{ row.batch_no || '-' }}</template></el-
table-column>
        <el-table-column label="效期" width="110"><template #default="{ row }">{{ row.expiry_date || '-' }}</template></el-
table-column>
        <el-table-column label="余量" width="80" align="center"><template
#default="{ row }">{{ row.recorded_quantity }}</template></el-table-column>
      </el-table>
      <template #footer>
        <el-button @click="onPrintView" :icon="Printer">打印盘库表</el-button>
        <el-button @click="viewVisible=false">关闭</el-button>
      </template>
    </el-dialog>
  </div>
</template>

<script setup>
import { ref, reactive, computed, onMounted, watch } from 'vue'
import { ElMessage, ElMessageBox } from 'element-plus'
import { Plus, Refresh, Printer } from '@element-plus/icons-vue'
import { listInventoryChecks, getInventoryCheck, createInventoryCheck, deleteInventoryCheck, getReagentTemplate,
listAllReagentItems } from '../../api/reagent'
import { useAuthStore } from '../../store/auth'
import { useReagentStore } from '../../store/reagent'
import { errText } from '../../utils/errText'
import { printHtml } from '../../utils/printHtml'
import LibraryTabs from '../../components/reagent/LibraryTabs.vue'

const auth = useAuthStore()
const reagentStore = useReagentStore()
const canWrite = computed(() => auth.canWrite('reagents'))
const checks = ref([]), total = ref(0), page = ref(1), pageSize = ref(20), loading = ref(false)
const dialogVisible = ref(false), submitting = ref(false)
const dialogSearch = ref('')
const categoryFilter = ref('')  // '' | 'reagent' | 'consumable' | 'control' | 'brand'
const brandTypeFilter = ref('')  // '' | 'reagent' | 'consumable' | 'control'（仅 brand 模式生效）

// 切走「按品牌」分类时清除类型子筛选，避免残留影响其他分类
watch(categoryFilter, (v) => { if (v !== 'brand') brandTypeFilter.value = '' })
const checkForm = ref({ check_date: '', check_type: '月末盘库', remark: '' })
const tpl = ref(null)
const quantities = reactive({})
const nameMap = ref({})
```

```
const viewVisible = ref(false), viewItems = ref([]), viewLibrary = ref('')

async function refresh() {
  loading.value = true
  try {
    const r = await listInventoryChecks({ library: reagentStore.library, page: page.value, page_size: pageSize.value })
    checks.value = r.items; total.value = r.total
  } catch (e) { ElMessage.error('加载失败：' + errText(e)) } finally { loading.value = false }
}

function allItems() {
  if (!tpl.value) return []
  const out = []
  for (const g of tpl.value.by_project) out.push(...g.items)
  for (const g of tpl.value.by_instrument) out.push(...g.items)
  out.push(...tpl.value.controls)
  return out
}
const totalEntries = computed(() => {
  if (!tpl.value) return 0
  if (categoryFilter.value === 'brand') return allItems().filter(it => matchItem(it) && matchType(it)).length
  let n = 0
  if (categoryFilter.value === '' || categoryFilter.value === 'reagent')
    for (const g of tpl.value.by_project) n += g.items.filter(matchItem).length
  if (categoryFilter.value === '' || categoryFilter.value === 'consumable')
    for (const g of tpl.value.by_instrument) n += g.items.filter(matchItem).length
  if ((categoryFilter.value === '' || categoryFilter.value === 'control') && tpl.value.controls)
    n += tpl.value.controls.filter(matchItem).length
  return n
})

/** 模糊匹配：试剂名/规格/编码/品牌 + 项目中文名/英文别名 */
function matchItem(it) {
  const kw = dialogSearch.value.trim().toLowerCase()
  if (!kw) return true
  // 试剂自身字段
  if ((it.name || '').toLowerCase().includes(kw)) return true
  if ((it.spec || '').toLowerCase().includes(kw)) return true
  if ((it.material_code || '').toLowerCase().includes(kw)) return true
  // 品牌（按品牌检索）
  if ((it.brand || '').toLowerCase().includes(kw)) return true
  // 项目名称 + 英文别名（逗号分隔，支持 ALT / ATIII 等）
  if (it.project_name && it.project_name.toLowerCase().includes(kw)) return true
  if (it.project_aliases) {
    for (const alias of it.project_aliases.split(','))
      if (alias.trim().toLowerCase().includes(kw)) return true
  }
  return false
}
```

```
/** 按品牌视图下的类型子筛选：试剂(含校准品)/耗材/质控品 */
function matchType(it) {
  const t = brandTypeFilter.value
  if (!t) return true
  if (t === 'reagent') return it.type === '试剂' || it.type === '校准品'
  if (t === 'consumable') return it.type === '耗材'
  if (t === 'control') return it.type === '质控品'
  return true
}

// 甲状腺功能项目统一归到「甲状腺」分组下展示
const THYROID_NAMES = ["甲状腺素", "三碘甲状腺原氨酸", "游离甲状腺素", "游离三碘甲状腺原氨酸", "促甲状腺激素", "抗甲状腺过氧化物酶抗体", "甲状腺球蛋白", "促甲状腺激素受体抗体", "抗甲状腺球蛋白抗体"]
function isThyroid(name) {
  if (!name) return false
  if (THYROID_NAMES.includes(name)) return true
  return name.includes('甲状腺')
}
function groupProjects(projects) {
  const thyroid = []
  const others = []
  for (const g of projects) {
    if (isThyroid(g.test_item_name)) thyroid.push(...g.items)
    else others.push(g)
  }
  if (thyroid.length) others.unshift({ test_item_id: null, test_item_name: '甲状腺', test_item_aliases: '', items: thyroid })
  return others
}
const filteredProjects = computed(() => {
  if (!tpl.value) return []
  if (categoryFilter.value && categoryFilter.value !== 'reagent') return []
  const filtered = tpl.value.by_project.map(g => ({ ...g, items: g.items.filter(matchItem) })).filter(g => g.items.length)
  return groupProjects(filtered)
})
const filteredInstruments = computed(() => {
  if (!tpl.value) return []
  if (categoryFilter.value && categoryFilter.value !== 'consumable') return []
  return tpl.value.by_instrument.map(g => ({ ...g, items: g.items.filter(matchItem) })).filter(g => g.items.length)
})
const filteredControls = computed(() => {
  if (!tpl.value || !tpl.value.controls) return []
  if (categoryFilter.value && categoryFilter.value !== 'control') return []
  return tpl.value.controls.filter(matchItem)
})
const filteredByBrand = computed(() => {
  if (!tpl.value || categoryFilter.value !== 'brand') return []
  const seen = new Set()  // 按 item_id 去重：同品牌多仪器共用的耗材（如反应杯）只显示一次
  const map = {}
  for (const it of allItems()) {
    if (seen.has(it.item_id)) continue
    if (!matchItem(it) || !matchType(it)) continue
```

```
    seen.add(it.item_id)
    const b = it.brand || '未标注品牌'
    if (!map[b]) map[b] = []
    map[b].push(it)
  }
  return Object.keys(map)
    .sort((a, b) => {
      if (a === '未标注品牌') return 1
      if (b === '未标注品牌') return -1
      return a.localeCompare(b, 'zh-CN')
    })
    .map(brand => ({ brand, items: map[brand] }))
})

async function onNewCheck() {
  dialogSearch.value = ''
  categoryFilter.value = ''
  brandTypeFilter.value = ''
  checkForm.value = { check_date: new Date().toISOString().slice(0,10), check_type: '月末盘库', remark: '' }
  dialogVisible.value = true
  submitting.value = true
  try {
    const r = await getReagentTemplate({ library: reagentStore.library })
    tpl.value = r[reagentStore.library] || { by_project: [], by_instrument: [], controls: [] }
    for (const it of allItems()) quantities[it.item_id] = it.current_stock
  } catch (e) {
    ElMessage.error('加载模板失败：' + errText(e))
  } finally { submitting.value = false }
}

function buildSections(useRecorded) {
  const secs = []
  for (const g of groupProjects(tpl.value.by_project)) {
    secs.push({ heading: '项目：' + g.test_item_name, items: g.items.map(it => ({
      name: it.name, spec: it.spec, unit: it.unit, material_code: it.material_code,
      brand: it.brand,
      current: it.current_stock,
      qty: useRecorded ? (quantities[it.item_id] || 0) : '',
    })) })
  }
  for (const g of tpl.value.by_instrument) {
    const extra = g.instruments.length ? `（${g.instruments.join('、')}）` : ''
    secs.push({ heading: '仪器：' + g.group + extra, items: g.items.map(it => ({
      name: it.name, spec: it.spec, unit: it.unit, material_code: it.material_code,
      brand: it.brand,
      current: it.current_stock,
      qty: useRecorded ? (quantities[it.item_id] || 0) : '',
    })) })
  }
  if (tpl.value.controls.length) {
```

```
    secs.push({ heading: '质控品（单独）', items: tpl.value.controls.map(it => ({
      name: it.name, spec: it.spec, unit: it.unit, material_code: it.material_code,
      brand: it.brand,
      current: it.current_stock,
      qty: useRecorded ? (quantities[it.item_id] || 0) : '',
    })) })
  }
  return secs
}

function sectionHtml(secs) {
  let h = ''
  for (const s of secs) {
    h += `<h3>${s.heading}</h3>`
    h += '<table><thead><tr><th>名称</th><th>规格</th><th>品牌</th><th>材料编码</th><th>单位</th><th class="num">当前库存
</th><th class="num">盘点余量</th></tr></thead><tbody>'
    for (const it of s.items) {
      h += `<tr><td>${it.name || ''}</td><td>${it.spec || ''}</td><td>${it.brand || ''}</td><td>${it.material_code ||
''}</td><td>${it.unit || ''}</td><td class="num">${it.current}</td><td class="num">${it.qty}</td></tr>`
    }
    h += '</tbody></table>'
  }
  return h
}

function onPrintBlank() {
  if (!tpl.value) return
  const html = sectionHtml(buildSections(false))
  printHtml(`盘库空白录入页（${reagentStore.library}）`,
    `<h2>试剂盘库录入页（空白）</h2><div class="meta">责任库：${reagentStore.library}　打印日期：${new
Date().toLocaleDateString('zh-CN')}</div>${html}`)
}

async function onSubmit() {
  if (!checkForm.value.check_date) { ElMessage.warning('请选择盘库日期'); return }
  submitting.value = true
  try {
    const items = allItems().map(it => ({
      item_id: it.item_id, batch_no: '', expiry_date: null,
      recorded_quantity: Number(quantities[it.item_id] || 0),
    }))
    await createInventoryCheck({ library: reagentStore.library, ...checkForm.value, items })
    ElMessage.success('盘库成功，实时库存已更新')
    dialogVisible.value = false; refresh()
  } catch (e) { ElMessage.error('盘库失败：' + errText(e)) } finally { submitting.value = false }
}

async function onView(row) {
  try {
    const r = await getInventoryCheck(row.id)
    if (Object.keys(nameMap.value).length === 0) {
      // 加载全库名称映射（含停用项），避免因责任库不同导致跨库盘库解析不到名称而回退成 (id=…)
      const all = await listAllReagentItems()
```

```
      for (const it of all) nameMap.value[it.id] = it
    }
    // 只展示实际盘点过且有余量的条目；数量为 0 / 空 视为“未盘”，详情与打印均不显示
    viewItems.value = (r.items || [])
      .filter(i => (i.recorded_quantity ?? 0) > 0)
      .map(i => ({
        ...i,
        _name: nameMap.value[i.item_id]?.name || '',
        _spec: nameMap.value[i.item_id]?.spec || '',
      }))
    viewLibrary.value = row.library || reagentStore.library
    viewVisible.value = true
  } catch (e) { ElMessage.error('加载详情失败：' + errText(e)) }
}

function onPrintView() {
  let h = '<table><thead><tr><th>名称</th><th>规格</th><th>批号</th><th>效期</th><th class="num">余量
</th></tr></thead><tbody>'
  for (const it of viewItems.value) {
    const qty = (it.recorded_quantity === null || it.recorded_quantity === undefined) ? '-' : it.recorded_quantity
    h += `<tr><td>${it._name || it.item_id}</td><td>${it._spec || ''}</td><td>${it.batch_no || ''}</td><td>${it.expiry_date
|| ''}</td><td class="num">${qty}</td></tr>`
  }
  h += '</tbody></table>'
  printHtml(`盘库表（${viewLibrary.value}）`,
    `<h2>试剂盘库表</h2><div class="meta">责任库：${viewLibrary.value}　打印日期：${new Date().toLocaleDateString('zh-
CN')}</div>${h}`)
}

function onPrintCheck(row) { onView(row).then(() => { if (viewVisible.value) onPrintView() }) }

async function onDeleteCheck(row) {
  await ElMessageBox.confirm(
    `确认删除「${row.check_date} ${row.check_type}」盘库记录？该操作不可恢复。`,
    '删除盘库', { type: 'warning' }
  )
  await deleteInventoryCheck(row.id); ElMessage.success('已删除'); refresh()
}

onMounted(refresh)
</script>
<style scoped>
.page { padding: 16px 20px 0; display: flex; flex-direction: column; height: 100%; }
.page-header { margin-bottom: 8px; }
.title { margin: 0; font-size: 20px; }
.sub { margin: 4px 0 0; color: #64748b; font-size: 13px; }
.toolbar { display: flex; gap: 10px; align-items: center; margin: 8px 0 12px; flex-wrap: wrap; }
.pager { margin: 10px 0 16px; display: flex; justify-content: flex-end; }
.muted { color: #94a3b8; }
.entry-scroll { max-height: 56vh; overflow: auto; border: 1px solid #e2e8f0; border-radius: 6px; padding: 8px; }
.grp-title { margin: 14px 0 6px; font-size: 14px; color: #0f172a; border-left: 4px solid #2563eb; padding-left: 8px; }
.grp-title:first-child { margin-top: 4px; }
</style>
```

```
================================================================================
# 文件: views/reagent/ReagentOrders.vue  (D:\workbuddyprojects\网页版-生免速查工具
\frontend\src\views\reagent\ReagentOrders.vue)
================================================================================

<template>
  <div class="page">
    <div class="page-header">
      <h2 class="title">订购管理</h2>
      <p class="sub">按项目显示试剂、按仪器显示耗材、质控品单独列示。录入需订购数量，生成订购表可导出 Excel 或打印（含材料编码）。
</p>
    </div>
    <LibraryTabs @change="refresh" />

    <div class="toolbar">
      <el-button type="primary" :icon="Plus" @click="onNewOrder" v-if="canWrite">新建订购</el-button>
      <el-button :icon="Refresh" @click="refresh">刷新</el-button>
    </div>
    <el-table v-loading="loading" :data="orders" border stripe height="calc(100vh - 360px)">
      <el-table-column type="index" width="50" />
      <el-table-column prop="library" label="责任库" width="100">
        <template #default="{ row }">
          <el-tag v-if="row.library" size="small" :type="row.library === '免疫' ? 'warning' :
'success'">{{ row.library }}</el-tag>
          <span v-else class="muted">—</span>
        </template>
      </el-table-column>
      <el-table-column prop="order_no" label="订单号" width="180" />
      <el-table-column prop="order_date" label="日期" width="110" />
      <el-table-column prop="order_type" label="类型" width="100" />
      <el-table-column label="状态" width="90">
        <template #default="{ row }">
          <el-tag :type="row.status === '完成' ? 'success' : row.status === '已提交' ? 'primary' : 'info'"
size="small">{{ row.status }}</el-tag>
        </template>
      </el-table-column>
      <el-table-column label="细项数" width="70" align="center">
        <template #default="{ row }">{{ row.items?.length || 0 }}</template>
      </el-table-column>
      <el-table-column prop="operator" label="操作人" width="120" />
      <el-table-column prop="remark" label="备注" min-width="160" show-overflow-tooltip />
      <el-table-column prop="created_by" label="创建人" width="100" />
      <el-table-column label="操作" width="230" fixed="right" v-if="canWrite">
        <template #default="{ row }">
          <el-button size="small" link type="primary" @click="onExport(row)">导出</el-button>
          <el-button size="small" link type="primary" @click="onPrint(row)">打印</el-button>
          <el-button v-if="canEditRow(row)" size="small" link type="primary" @click="onEdit(row)">编辑</el-button>
          <el-button v-if="!row.is_confirmed" size="small" link type="success" @click="onConfirm(row)">确认</el-button>
        </template>
      </el-table-column>
    </el-table>
    <el-pagination class="pager" v-model:current-page="page" v-model:page-size="pageSize"
      :total="total" :page-sizes="[20,50]" layout="total, sizes, prev, pager, next"
      @current-change="refresh" @size-change="page=1; refresh()" />

    <!-- 新建/编辑订购弹窗 -->
    <el-dialog v-model="dialogVisible" :title="(editingId ? '编辑订购' : '新建订购') + ` （${reagentStore.library}）`"
width="min(960px, 96vw)" top="3vh">
```

```
      <el-form :model="orderForm" label-width="80px" size="small">
        <el-row :gutter="12">
          <el-col :span="8"><el-form-item label="订单号"><el-input v-model="orderForm.order_no" placeholder="ORD-202607-001"
/></el-form-item></el-col>
          <el-col :span="8"><el-form-item label="日期"><el-date-picker v-model="orderForm.order_date" type="date" value-
format="YYYY-MM-DD" style="width:100%" /></el-form-item></el-col>
          <el-col :span="8"><el-form-item label="类型"><el-select v-model="orderForm.order_type" style="width:100%">
            <el-option label="月初订购" value="月初订购" /><el-option label="加订" value="加订" />
          </el-select></el-form-item></el-col>
        </el-row>
        <el-form-item label="备注"><el-input v-model="orderForm.remark" /></el-form-item>
      </el-form>
      <div style="margin-bottom:8px;display:flex;gap:8px;align-items:center;flex-wrap:wrap">
        <el-select v-model="categoryFilter" placeholder="全部分类" clearable style="width:130px" size="default">
          <el-option label="试剂 / 校准品" value="reagent" />
          <el-option label="耗材" value="consumable" />
          <el-option label="质控品" value="control" />
        </el-select>
        <el-input v-model="dialogSearch" placeholder="模糊检索试剂/规格/编码/项目名(含英文别名如 ALT)..." clearable
style="width:340px" />
        <span class="muted" style="font-size:12px">共 {{ totalEntries }} 项</span>
      </div>

      <div class="entry-scroll">
        <template v-for="grp in filteredProjects" :key="'p'+grp.test_item_id">
          <h4 class="grp-title">项目：{{ grp.test_item_name }}</h4>
          <div class="entry-card-list">
            <div class="entry-card" v-for="it in grp.items" :key="it.item_id">
              <div class="entry-card-name">
                <div>{{ it.name }} <span class="muted">{{ it.spec }}</span></div>
                <div class="card-stock">实时库存：{{ it.current_stock }} {{ it.unit }}<span v-if="it.min_stock">（最低
{{ it.min_stock }}）</span></div>
              </div>
              <div class="entry-card-input">
                <span class="muted">订购数量</span>
                <el-input-number v-model="quantities[it.item_id]" :min="0" size="small" controls-position="right" />
              </div>
            </div>
          </div>
        </template>
        <template v-for="grp in filteredInstruments" :key="'i'+grp.group">
          <h4 class="grp-title">仪器：{{ grp.group }} <span class="muted" v-if="grp.instruments.length">
（{{ grp.instruments.join('、') }}）</span></h4>
          <div class="entry-card-list">
            <div class="entry-card" v-for="it in grp.items" :key="it.item_id">
              <div class="entry-card-name">
                <div>{{ it.name }} <span class="muted">{{ it.spec }}</span></div>
                <div class="card-stock">实时库存：{{ it.current_stock }} {{ it.unit }}<span v-if="it.min_stock">（最低
{{ it.min_stock }}）</span></div>
              </div>
              <div class="entry-card-input">
                <span class="muted">订购数量</span>
                <el-input-number v-model="quantities[it.item_id]" :min="0" size="small" controls-position="right" />
              </div>
            </div>
          </div>
```

```
        </template>
        <template v-if="filteredControls.length">
          <h4 class="grp-title">质控品（单独）</h4>
          <div class="entry-card-list">
            <div class="entry-card" v-for="it in filteredControls" :key="it.item_id">
              <div class="entry-card-name">
                <div>{{ it.name }} <span class="muted">{{ it.spec }}</span></div>
                <div class="card-stock">实时库存：{{ it.current_stock }} {{ it.unit }}<span v-if="it.min_stock">（最低
{{ it.min_stock }}）</span></div>
              </div>
              <div class="entry-card-input">
                <span class="muted">订购数量</span>
                <el-input-number v-model="quantities[it.item_id]" :min="0" size="small" controls-position="right" />
              </div>
            </div>
          </div>
        </template>
        <el-empty v-if="totalEntries === 0" description="该分类/检索条件下无数据" />
      </div>

      <template #footer>
        <el-button @click="dialogVisible=false">取消</el-button>
        <el-button type="primary" :loading="submitting" @click="onSubmit">保存</el-button>
      </template>
    </el-dialog>
  </div>
</template>

<script setup>
import { ref, reactive, computed, onMounted } from 'vue'
import { ElMessage, ElMessageBox } from 'element-plus'
import { Plus, Refresh } from '@element-plus/icons-vue'
import {
  listReagentOrders, createReagentOrder, updateReagentOrder,
  exportOrderForm, getReagentTemplate, listAllReagentItems,
  getNextOrderNo, confirmReagentOrder,
} from '../../api/reagent'
import { useAuthStore } from '../../store/auth'
import { useReagentStore } from '../../store/reagent'
import { errText } from '../../utils/errText'
import { printHtml } from '../../utils/printHtml'
import LibraryTabs from '../../components/reagent/LibraryTabs.vue'

const auth = useAuthStore()
const reagentStore = useReagentStore()
const canWrite = computed(() => auth.canWrite('reagent-orders'))
// 编辑权限：管理员/试剂管理员可改任意；试剂配送仅能改自己创建的；已确认的不可改（后端再兜底）
function canEditRow(row) {
  if (auth.isAdmin) return true
  return !!(row.created_by && row.created_by === auth.user?.username)
}
```

```
const orders = ref([]), total = ref(0), page = ref(1), pageSize = ref(20), loading = ref(false)
const dialogVisible = ref(false), editingId = ref(null), submitting = ref(false)
const dialogSearch = ref('')
const categoryFilter = ref('')
const orderForm = ref({ order_no: '', order_date: new Date().toISOString().slice(0,10), order_type: '月初订购', remark: '' })
const tpl = ref(null)
const quantities = reactive({})

async function refresh() {
  loading.value = true
  try {
    const r = await listReagentOrders({ library: reagentStore.library, page: page.value, page_size: pageSize.value })
    orders.value = r.items; total.value = r.total
  } catch (e) { ElMessage.error('加载失败：' + errText(e)) } finally { loading.value = false }
}

function allItems() {
  if (!tpl.value) return []
  const out = []
  for (const g of tpl.value.by_project) out.push(...g.items)
  for (const g of tpl.value.by_instrument) out.push(...g.items)
  out.push(...tpl.value.controls)
  return out
}
const totalEntries = computed(() => {
  if (!tpl.value) return 0
  let n = 0
  if (categoryFilter.value === '' || categoryFilter.value === 'reagent')
    for (const g of tpl.value.by_project) n += g.items.filter(matchItem).length
  if (categoryFilter.value === '' || categoryFilter.value === 'consumable')
    for (const g of tpl.value.by_instrument) n += g.items.filter(matchItem).length
  if ((categoryFilter.value === '' || categoryFilter.value === 'control') && tpl.value.controls)
    n += tpl.value.controls.filter(matchItem).length
  return n
})

/** 模糊匹配：试剂名/规格/编码 + 项目中文名/英文别名 */
function matchItem(it) {
  const kw = dialogSearch.value.trim().toLowerCase()
  if (!kw) return true
  if ((it.name || '').toLowerCase().includes(kw)) return true
  if ((it.spec || '').toLowerCase().includes(kw)) return true
  if ((it.material_code || '').toLowerCase().includes(kw)) return true
  if (it.project_name && it.project_name.toLowerCase().includes(kw)) return true
  if (it.project_aliases) {
    for (const alias of it.project_aliases.split(','))
      if (alias.trim().toLowerCase().includes(kw)) return true
  }
  return false
}
```

```
const filteredProjects = computed(() => {
  if (!tpl.value) return []
  if (categoryFilter.value && categoryFilter.value !== 'reagent') return []
  return tpl.value.by_project.map(g => ({ ...g, items: g.items.filter(matchItem) })).filter(g => g.items.length)
})
const filteredInstruments = computed(() => {
  if (!tpl.value) return []
  if (categoryFilter.value && categoryFilter.value !== 'consumable') return []
  return tpl.value.by_instrument.map(g => ({ ...g, items: g.items.filter(matchItem) })).filter(g => g.items.length)
})
const filteredControls = computed(() => {
  if (!tpl.value || !tpl.value.controls) return []
  if (categoryFilter.value && categoryFilter.value !== 'control') return []
  return tpl.value.controls.filter(matchItem)
})

async function loadTemplate() {
  const r = await getReagentTemplate({ library: reagentStore.library })
  tpl.value = r[reagentStore.library] || { by_project: [], by_instrument: [], controls: [] }
  for (const it of allItems()) if (quantities[it.item_id] == null) quantities[it.item_id] = 0
}

async function onNewOrder() {
  dialogSearch.value = ''
  categoryFilter.value = ''
  let nextNo = ''
  try { const r = await getNextOrderNo(); nextNo = r.order_no || '' } catch (_) {}
  orderForm.value = { order_no: nextNo, order_date: new Date().toISOString().slice(0,10), order_type: '月初订购', remark:
'' }
  editingId.value = null
  for (const k in quantities) delete quantities[k]
  dialogVisible.value = true
  loadTemplate().catch(e => ElMessage.error('加载模板失败：' + errText(e)))
}

function onEdit(row) {
  dialogSearch.value = ''
  categoryFilter.value = ''
  editingId.value = row.id
  orderForm.value = { order_no: row.order_no, order_date: row.order_date, order_type: row.order_type, remark: row.remark ||
'' }
  for (const k in quantities) delete quantities[k]
  dialogVisible.value = true
  loadTemplate().then(() => {
    for (const it of (row.items || [])) quantities[it.item_id] = it.ordered_quantity || 0
  }).catch(e => ElMessage.error('加载模板失败：' + errText(e)))
}

async function onSubmit() {
  if (!orderForm.value.order_no) { ElMessage.warning('请填写订单号'); return }
  submitting.value = true
  try {
```

```
    const items = allItems()
      .filter(it => Number(quantities[it.item_id] || 0) > 0)
      .map(it => ({ item_id: it.item_id, ordered_quantity: Number(quantities[it.item_id] || 0) }))
    const data = { library: reagentStore.library, ...orderForm.value, status: '草稿', items }
    if (editingId.value) await updateReagentOrder(editingId.value, data)
    else await createReagentOrder(data)
    ElMessage.success('已保存'); dialogVisible.value = false; refresh()
  } catch (e) { ElMessage.error('保存失败：' + errText(e)) } finally { submitting.value = false }
}

async function onConfirm(row) {
  try {
    await ElMessageBox.confirm(`确认提交订购单「${row.order_no}」？提交后将不可再修改。`, '确认订购单', { type: 'warning' })
    await confirmReagentOrder(row.id)
    ElMessage.success('已确认提交'); refresh()
  } catch (e) { if (e !== 'cancel') ElMessage.error('确认失败：' + errText(e)) }
}

function onExport(row) {
  exportOrderForm(row.id).then((blob) => {
    const url = window.URL.createObjectURL(new Blob([blob]))
    const a = document.createElement('a')
    a.href = url
    a.download = `设备科订购表_${row.order_no}.xlsx`
    document.body.appendChild(a)
    a.click()
    document.body.removeChild(a)
    window.URL.revokeObjectURL(url)
    ElMessage.success('已导出订购表')
  }).catch(() => ElMessage.error('导出失败'))
}

async function onPrint(row) {
  try {
    const r = await listAllReagentItems({ library: row.library || reagentStore.library })
    const nm = {}
    for (const it of r) nm[it.id] = it
    let h = '<table><thead><tr><th>材料编码</th><th>名称</th><th>规格</th><th>单位</th><th class="num">订购数量
</th></tr></thead><tbody>'
    for (const it of (row.items || [])) {
      const m = nm[it.item_id] || {}
      h += `<tr><td>${m.material_code || ''}</td><td>${m.name || it.item_id}</td><td>${m.spec || ''}</td><td>${m.unit ||
''}</td><td class="num">${it.ordered_quantity}</td></tr>`
    }
    h += '</tbody></table>'
    printHtml(`试剂订购表 ${row.order_no}`,
      `<h2>试剂订购表</h2><div class="meta">订单号：${row.order_no}　日期：${row.order_date}　责任库：${row.library || ''}　类
型：${row.order_type}</div>${h}`)
  } catch (e) { ElMessage.error('打印失败：' + errText(e)) }
}

onMounted(refresh)
</script>
```

```
<style scoped>
.page { padding: 16px 20px 0; display: flex; flex-direction: column; height: 100%; }
.page-header { margin-bottom: 8px; }
.title { margin: 0; font-size: 20px; }
.sub { margin: 4px 0 0; color: #64748b; font-size: 13px; }
.toolbar { display: flex; gap: 10px; align-items: center; margin: 8px 0 12px; flex-wrap: wrap; }
.pager { margin: 10px 0 16px; display: flex; justify-content: flex-end; }
.muted { color: #94a3b8; }
.entry-scroll { max-height: 56vh; overflow: auto; border: 1px solid #e2e8f0; border-radius: 6px; padding: 8px; }
.grp-title { margin: 14px 0 6px; font-size: 14px; color: #0f172a; border-left: 4px solid #2563eb; padding-left: 8px; }
.grp-title:first-child { margin-top: 4px; }
/* 移动端卡片式录入：名称换行 + 数量输入在右，避免横向滑动 */
.entry-card-list { display: flex; flex-direction: column; gap: 6px; margin-bottom: 10px; }
.entry-card { display: flex; align-items: center; justify-content: space-between; gap: 10px; padding: 8px 10px; border: 1px
solid #e2e8f0; border-radius: 6px; background: #fff; }
.entry-card-name { flex: 1; min-width: 0; font-size: 13px; color: #0f172a; line-height: 1.4; word-break: break-word; }
.card-stock { font-size: 12px; color: #2563eb; margin-top: 2px; }
.entry-card-input { display: flex; align-items: center; gap: 6px; white-space: nowrap; flex-shrink: 0; }
@media (max-width: 480px) {
  .entry-card { flex-direction: column; align-items: stretch; }
  .entry-card-input { justify-content: space-between; }
}
</style>

======================================================================
# 文件: views/reagent/ReagentReceivings.vue  (D:\workbuddyprojects\网页版-生免速查工具
\frontend\src\views\reagent\ReagentReceivings.vue)
======================================================================

<template>
  <div class="page">
    <div class="page-header">
      <h2 class="title">到货接收</h2>
      <p class="sub">试剂送货到货录入，确认接收后才会进入实时库存。接收人默认为当前登录人员。</p>
    </div>
    <LibraryTabs @change="refresh" />
    <div class="toolbar">
      <el-button type="primary" :icon="Plus" @click="onNewReceiving" v-if="canWrite">新增收货</el-button>
      <el-radio-group v-model="confirmedFilter" @change="refresh" size="small">
        <el-radio-button label="全部" value="" />
        <el-radio-button label="待确认" value="false" />
        <el-radio-button label="已确认" value="true" />
      </el-radio-group>
      <el-button :icon="Refresh" @click="refresh">刷新</el-button>
    </div>
    <el-table v-loading="loading" :data="receivings" border stripe height="calc(100vh - 360px)">
      <el-table-column type="index" width="50" />
      <el-table-column prop="receipt_no" label="收货单号" width="180" />
      <el-table-column prop="receipt_date" label="日期" width="110" />
      <el-table-column label="细项数" width="70" align="center">
        <template #default="{ row }">{{ row.items?.length || 0 }}</template>
      </el-table-column>
      <el-table-column prop="delivery_person" label="送货人" width="100" />
      <el-table-column prop="receiver" label="接收人" width="100" />
      <el-table-column prop="created_by" label="创建人" width="100" />
      <el-table-column label="状态" width="160">
```

```
        <template #default="{ row }">
          <el-tag :type="row.is_confirmed ? 'success' : 'warning'" size="small">
            {{ row.is_confirmed ? '已确认' : '待确认' }}
          </el-tag>
          <div v-if="row.is_confirmed" class="sub-text">
            {{ row.confirmed_by }} · {{ fmt(row.confirmed_at) }}
          </div>
        </template>
      </el-table-column>
      <el-table-column prop="remark" label="备注" min-width="140" show-overflow-tooltip />
      <el-table-column label="操作" width="240" fixed="right">
        <template #default="{ row }">
          <el-button size="small" link type="primary" @click="onView(row)">详情</el-button>
          <el-button size="small" link type="success" @click="onConfirm(row)"
            v-if="!row.is_confirmed" :disabled="!canConfirm(row)">确认接收</el-button>
          <el-button size="small" link type="warning" @click="onEdit(row)"
            v-if="canEdit(row)">编辑</el-button>
          <el-button size="small" link type="danger" @click="onDelete(row)"
            v-if="auth.isAdmin">删除</el-button>
          <el-button size="small" link type="info" @click="onPrint(row)">打印</el-button>
        </template>
      </el-table-column>
    </el-table>
    <el-pagination class="pager" v-model:current-page="page" v-model:page-size="pageSize"
      :total="total" :page-sizes="[20,50]" layout="total, sizes, prev, pager, next"
      @current-change="refresh" @size-change="page=1; refresh()" />

    <!-- 新建/编辑收货弹窗 -->
    <el-dialog v-model="dialogVisible" :title="editingId ? '编辑收货' : '新增收货'" width="820px">
      <el-form :model="form" label-width="80px" size="small">
        <el-row :gutter="12">
          <el-col :span="8"><el-form-item label="收货单号"><el-input v-model="form.receipt_no" placeholder="RCV-202607-001"
/></el-form-item></el-col>
          <el-col :span="8"><el-form-item label="日期"><el-date-picker v-model="form.receipt_date" type="date" value-
format="YYYY-MM-DD" style="width:100%" /></el-form-item></el-col>
          <el-col :span="8"><el-form-item label="送货人"><el-input v-model="form.delivery_person" /></el-form-item></el-col>
        </el-row>
        <el-row :gutter="12">
          <el-col :span="8"><el-form-item label="接收人"><el-input v-model="form.receiver" placeholder="确认接收时自动填入当前
登录人" /></el-form-item></el-col>
        </el-row>
        <el-form-item label="备注"><el-input v-model="form.remark" /></el-form-item>
      </el-form>
      <div style="margin-bottom:8px;display:flex;gap:8px;align-items:center">
        <span class="label">搜索添加试剂：</span>
        <el-select v-model="searchSel" filterable remote :remote-method="onSearchItem" :loading="searching"
          placeholder="输入名称/品牌/编码模糊搜索" style="flex:1" value-key="id" clearable
          @change="onPickItem">
          <el-option v-for="o in searchResults" :key="o.id" :label="o.name + (o.spec ? ' ' + o.spec : '')" :value="o.id" />
        </el-select>
        <span class="hint">（模糊搜索后点选即可加入下方明细）</span>
      </div>
      <el-table :data="items" border size="small" max-height="360">
```

```
        <el-table-column label="试剂" min-width="200">
          <template #default="{ row }">
            <el-select v-model="row.item_id" filterable placeholder="选试剂" style="width:100%">
              <el-option v-for="it in allItems" :key="it.id" :label="it.name+(it.spec?' '+it.spec:'')" :value="it.id" />
            </el-select>
          </template>
        </el-table-column>
        <el-table-column label="批号" width="130"><template #default="{row}"><el-input v-model="row.batch_no" size="small"
/></template></el-table-column>
        <el-table-column label="效期" width="120"><template #default="{row}"><el-date-picker v-model="row.expiry_date"
type="date" value-format="YYYY-MM-DD" size="small" style="width:100%" /></template></el-table-column>
        <el-table-column label="数量" width="90"><template #default="{row}"><el-input-number v-model="row.quantity" :min="1"
size="small" style="width:90px" /></template></el-table-column>
        <el-table-column width="50"><template #default="{row,$index}"><el-button link type="danger" :icon="Delete"
@click="items.splice($index,1)" /></template></el-table-column>
      </el-table>
      <template #footer>
        <el-button @click="dialogVisible=false">取消</el-button>
        <el-button type="primary" :loading="submitting" @click="onSubmit">保存</el-button>
      </template>
    </el-dialog>

    <!-- 打印收货单 -->
    <el-dialog v-model="printVisible" title="收货单打印预览" width="720px" class="no-print-dialog">
      <div id="print-area" class="receipt">
        <h2 class="receipt-title">试剂到货接收单</h2>
        <table class="receipt-head">
          <tr>
            <td>收货单号：{{ printData.receipt_no }}</td>
            <td>日期：{{ printData.receipt_date }}</td>
          </tr>
          <tr>
            <td>送货人：{{ printData.delivery_person || '—' }}</td>
            <td>接收人：{{ printData.receiver || '—' }}</td>
          </tr>
          <tr>
            <td>状态：{{ printData.is_confirmed ? '已确认' : '待确认' }}</td>
            <td>创建人：{{ printData.created_by || '—' }}</td>
          </tr>
          <tr v-if="printData.is_confirmed">
            <td colspan="2">确认：{{ printData.confirmed_by }} · {{ fmt(printData.confirmed_at) }}</td>
          </tr>
          <tr>
            <td colspan="2">备注：{{ printData.remark || '—' }}</td>
          </tr>
        </table>
        <table class="receipt-items">
          <thead>
            <tr><th>序号</th><th>试剂名称</th><th>批号</th><th>效期</th><th>数量</th></tr>
          </thead>
          <tbody>
            <tr v-for="(it, i) in printData.items || []" :key="it.id">
              <td>{{ i + 1 }}</td>
              <td>{{ itemName(it.item_id) }}</td>
```

```
              <td>{{ it.batch_no || '—' }}</td>
              <td>{{ it.expiry_date || '—' }}</td>
              <td>{{ it.quantity }}</td>
            </tr>
          </tbody>
        </table>
        <div class="receipt-foot">
          <span>收货人签字：_____</span>
          <span>日期：_____年____月____日</span>
        </div>
      </div>
      <template #footer>
        <el-button @click="printVisible=false">关闭</el-button>
        <el-button type="primary" :icon="Printer" @click="doPrint">打印</el-button>
      </template>
    </el-dialog>
  </div>
</template>

<script setup>
import { ref, computed, onMounted } from 'vue'
import { ElMessage, ElMessageBox } from 'element-plus'
import { Plus, Refresh, Delete, Printer } from '@element-plus/icons-vue'
import {
  listReagentReceivings, createReagentReceiving, getReagentReceiving,
  updateReagentReceiving, confirmReagentReceiving, deleteReagentReceiving,
  listAllReagentItems, listReagentItems, getReagentItem,
} from '../../api/reagent'
import { useAuthStore } from '../../store/auth'
import { useReagentStore } from '../../store/reagent'
import { errText } from '../../utils/errText'
import LibraryTabs from '../../components/reagent/LibraryTabs.vue'

const auth = useAuthStore()
const reagentStore = useReagentStore()
const canWrite = computed(() => auth.canWrite('reagent-receivings'))
const currentUser = computed(() => auth.user?.username || '')

const receivings = ref([]), total = ref(0), page = ref(1), pageSize = ref(20), loading = ref(false)
const confirmedFilter = ref('')
const dialogVisible = ref(false), submitting = ref(false), editingId = ref(null)
const allItems = ref([]), searchSel = ref(''), searchResults = ref([]), searching = ref(false)
const loadedLib = ref('')
const form = ref({ receipt_no: '', receipt_date: '', delivery_person: '', receiver: '', remark: '' })
const items = ref([])
const printVisible = ref(false), printData = ref({})

function fmt(v) {
  if (!v) return ''
  return String(v).slice(0, 16).replace('T', ' ')
```

```
}
function itemName(id) {
  const it = allItems.value.find(x => x.id === id) || searchResults.value.find(x => x.id === id)
  return it ? (it.name + (it.spec ? ' ' + it.spec : '')) : ('ID#' + id)
}

async function refresh() {
  // 切库/筛选/翻页时清空试剂名映射缓存，避免下次 loadItems 早退导致切库后显示 ID
  allItems.value = []
  loadedLib.value = ''
  loading.value = true
  try {
    const params = { library: reagentStore.library, page: page.value, page_size: pageSize.value }
    if (confirmedFilter.value !== '') params.confirmed = confirmedFilter.value === 'true'
    const r = await listReagentReceivings(params)
    receivings.value = r.items; total.value = r.total
  } catch (e) { ElMessage.error('加载失败：' + errText(e)) } finally { loading.value = false }
}
async function loadItems() {
  // 已按当前库加载过则跳过；切库后 loadedLib 已被 refresh 清空，会重新拉取
  if (allItems.value.length && loadedLib.value === reagentStore.library) return
  const all = await listAllReagentItems({ library: reagentStore.library })
  allItems.value = all
  loadedLib.value = reagentStore.library
}
function onNewReceiving() {
  loadItems()
  editingId.value = null
  form.value = {
    receipt_no: 'RCV-' + new Date().toISOString().slice(0, 7) + '-' + String(Date.now()).slice(-3),
    receipt_date: new Date().toISOString().slice(0, 10),
    delivery_person: '',     receiver: '', remark: '',
  }
  items.value = []; searchResults.value = []; searchSel.value = ''
  dialogVisible.value = true
}
async function onEdit(row) {
  try {
    const r = await getReagentReceiving(row.id)
    editingId.value = row.id
    form.value = {
      receipt_no: r.receipt_no, receipt_date: r.receipt_date,
      delivery_person: r.delivery_person, receiver: r.receiver, remark: r.remark,
    }
    items.value = (r.items || []).map(it => ({
      item_id: it.item_id, batch_no: it.batch_no, expiry_date: it.expiry_date, quantity: it.quantity,
    }))
    await loadItems()
    // 补充当前库下拉中缺失的试剂（跨库混单或切库后，原明细试剂可能不在当前库）
    const missingIds = [...new Set(items.value
```

```
      .map(it => it.item_id)
      .filter(id => id && !allItems.value.some(x => x.id === id)))]
    if (missingIds.length) {
      const filled = await Promise.all(missingIds.map(id => getReagentItem(id).catch(() => null)))
      for (const it of filled) {
        if (it) allItems.value.push(it)
      }
    }
    dialogVisible.value = true
  } catch (e) { ElMessage.error('加载失败：' + errText(e)) }
}
async function onSearchItem(query) {
  const q = (query || '').trim()
  if (!q) { searchResults.value = []; return }
  searching.value = true
  try {
    const r = await listReagentItems({ q, library: reagentStore.library, show_inactive: true, page_size: 20 })
    searchResults.value = r.items || []
  } catch (_) { searchResults.value = [] } finally { searching.value = false }
}
function onPickItem(id) {
  if (!id) return
  const hit = searchResults.value.find(o => o.id === id)
  if (!hit) return
  if (items.value.find(c => c.item_id === id)) { ElMessage.info('已在明细中'); searchSel.value = ''; return }
  items.value.push({ item_id: id, batch_no: '', expiry_date: '', quantity: 1 })
  searchSel.value = ''
}
async function onSubmit() {
  if (!form.value.receipt_no) { ElMessage.warning('请填收货单号'); return }
  if (!items.value.length) { ElMessage.warning('请至少添加一项试剂'); return }
  submitting.value = true
  const payload = {
    ...form.value, order_id: null,
    items: items.value.filter(c => c.item_id).map(c => ({
      item_id: c.item_id, batch_no: c.batch_no || '', expiry_date: c.expiry_date || null, quantity: c.quantity || 0, remark:
'',
    })),
  }
  try {
    if (editingId.value) {
      await updateReagentReceiving(editingId.value, payload)
      ElMessage.success('已保存修改')
    } else {
      await createReagentReceiving(payload)
      ElMessage.success('收货单已保存（待确认接收后入库）')
    }
    dialogVisible.value = false; refresh()
  } catch (e) { ElMessage.error('提交失败：' + errText(e)) } finally { submitting.value = false }
}
async function onConfirm(row) {
```

```
  try {
    await ElMessageBox.confirm(`确认接收「${row.receipt_no}」？确认后试剂将进入实时库存。`, '确认接收', { type: 'warning' })
  } catch (_) { return }
  try {
    await confirmReagentReceiving(row.id)
    ElMessage.success('已确认接收，库存已更新')
    refresh()
  } catch (e) { ElMessage.error('确认失败：' + errText(e)) }
}
async function onView(row) {
  try {
    const r = await getReagentReceiving(row.id)
    const lines = (r.items || []).map((it, i) => `${i + 1}. ${itemName(it.item_id)} 批号:${it.batch_no || '—'} 效
期:${it.expiry_date || '—'} 数量:${it.quantity}`).join('\n')
    ElMessage.info(`收货单 ${r.receipt_no}（${r.is_confirmed ? '已确认' : '待确认'}）\n${lines || '无明细'}`)
  } catch (e) { ElMessage.error('加载失败：' + errText(e)) }
}
async function onPrint(row) {
  try {
    const r = await getReagentReceiving(row.id)
    printData.value = r
    await loadItems()
    printVisible.value = true
  } catch (e) { ElMessage.error('加载失败：' + errText(e)) }
}
function doPrint() { window.print() }

// 管理员删除收货单（已确认的会回退库存）
async function onDelete(row) {
  if (!auth.isAdmin) return
  try {
    await ElMessageBox.confirm(
      `确认删除收货单「${row.receipt_no}」？${row.is_confirmed ? '该单已确认，删除后将回退对应库存。' : ''}此操作不可恢复。`,
      '删除收货单', { type: 'warning', confirmButtonText: '删除', cancelButtonText: '取消' }
    )
  } catch (_) { return }
  try {
    await deleteReagentReceiving(row.id)
    ElMessage.success(`收货单 ${row.receipt_no} 已删除`)
    refresh()
  } catch (e) { ElMessage.error('删除失败：' + errText(e)) }
}

// 权限判定：试剂配送仅能改自己建的、且未确认的；管理员/试剂管理员可改任意未确认
function canEdit(row) {
  if (!canWrite.value) return false
  if (row.is_confirmed) return false
  if (auth.isAdmin) return true
  const roles = auth.myRoles
  if (roles.includes('reagent_manager')) return true
  // 试剂配送：仅本人
```

```
  return row.created_by === currentUser.value
}
function canConfirm() {
  if (auth.isAdmin) return true
  return auth.myRoles.includes('reagent_manager')
}
onMounted(refresh)
</script>
<style scoped>
.page { padding: 16px 20px 0; display: flex; flex-direction: column; height: 100%; }
.page-header { margin-bottom: 8px; }
.title { margin: 0; font-size: 20px; }
.sub { margin: 4px 0 0; color: #64748b; font-size: 13px; }
.toolbar { display: flex; gap: 10px; align-items: center; margin: 8px 0 12px; flex-wrap: wrap; }
.pager { margin: 10px 0 16px; display: flex; justify-content: flex-end; }
.sub-text { font-size: 11px; color: #94a3b8; margin-top: 2px; }
.label { font-size: 13px; color: #475569; }
.hint { font-size: 11px; color: #94a3b8; }
.receipt { padding: 8px 4px; }
.receipt-title { text-align: center; font-size: 18px; margin: 0 0 12px; }
.receipt-head { width: 100%; border-collapse: collapse; font-size: 13px; }
.receipt-head td { padding: 6px 4px; }
.receipt-items { width: 100%; border-collapse: collapse; margin-top: 12px; font-size: 13px; }
.receipt-items th, .receipt-items td { border: 1px solid #cbd5e1; padding: 6px 8px; text-align: left; }
.receipt-items th { background: #f1f5f9; }
.receipt-foot { display: flex; justify-content: space-between; margin-top: 24px; font-size: 13px; }
</style>
<style>
@media print {
  body * { visibility: hidden; }
  #print-area, #print-area * { visibility: visible; }
  #print-area { position: absolute; left: 0; top: 0; width: 100%; padding: 0 12px; }
  .no-print-dialog .el-dialog__header,
  .no-print-dialog .el-dialog__footer { display: none !important; }
}
</style>

========================================================================
# 文件: views/reagents/ReagentList.vue  (D:\workbuddyprojects\网页版-生免速查工具\frontend\src\views\reagents\ReagentList.vue)
========================================================================

<template>
  <div class="page">
    <CrudTable
      ref="crud"
      :columns="columns"
      :fetch="fetch"
      search-placeholder="搜索名称 / 批号 / 厂家 / 供应商..."
      :show-add="auth.canWrite('reagents')"
      :can-write="auth.canWrite('reagents')"
      @add="onAdd"
      @edit="onEdit"
      @delete="onDelete"
    />
```

```
    <EditDialog
      v-model="dialogVisible"
      :title="editingId ? '编辑试剂' : '新增试剂'"
      :form="form"
      :fields="fields"
      :rules="rules"
      :submitting="submitting"
      @submit="onSubmit"
    />
  </div>
</template>

<script setup>
import { ref, reactive } from 'vue'
import { ElMessage, ElMessageBox } from 'element-plus'
import CrudTable from '../../components/CrudTable.vue'
import EditDialog from '../../components/EditDialog.vue'
import { listReagents, createReagent, updateReagent, deleteReagent } from '../../api/reagents'
import { useAuthStore } from '../../store/auth'

const crud = ref(null)
const auth = useAuthStore()
const dialogVisible = ref(false)
const editingId = ref(null)
const submitting = ref(false)

const STATUS_OPTIONS = ['在库', '预警', '过期', '停用'].map((v) => ({ label: v, value: v }))

const fields = [
  { prop: 'name', label: '试剂名称' },
  { prop: 'brand', label: '品牌/厂家' },
  { prop: 'spec', label: '规格' },
  { prop: 'lot_no', label: '批号' },
  { prop: 'quantity', label: '库存数量' },
  { prop: 'unit', label: '单位', placeholder: '盒 / 支 / 瓶' },
  { prop: 'production_date', label: '生产日期', type: 'date' },
  { prop: 'expiry_date', label: '有效期至', type: 'date' },
  { prop: 'in_date', label: '入库日期', type: 'date' },
  { prop: 'supplier', label: '供应商' },
  { prop: 'storage_condition', label: '储存条件', placeholder: '如 2-8℃' },
  { prop: 'status', label: '状态', type: 'select', options: STATUS_OPTIONS },
  { prop: 'operator', label: '经手人' },
  { prop: 'remark', label: '备注', type: 'textarea' },
]

const rules = {
  name: [{ required: true, message: '请填写试剂名称', trigger: 'blur' }],
}

const columns = [
```

```
  { prop: 'name', label: '名称', width: 160 },
  { prop: 'brand', label: '厂家', width: 120 },
  { prop: 'spec', label: '规格', width: 110 },
  { prop: 'lot_no', label: '批号', width: 120 },
  { prop: 'quantity', label: '库存', width: 80 },
  { prop: 'unit', label: '单位', width: 70 },
  { prop: 'expiry_date', label: '有效期至', width: 110 },
  { prop: 'supplier', label: '供应商', width: 140 },
  {
    prop: 'status', label: '状态', width: 90,
    formatter: (row) => {
      const map = { 在库: 'success', 预警: 'warning', 过期: 'danger', 停用: 'info' }
      const t = map[row.status] || 'info'
      return `<el-tag type="${t}" size="small">${row.status || '-'}</el-tag>`
    },
  },
]

const emptyForm = () => ({
  name: '', brand: '', spec: '', lot_no: '', quantity: '', unit: '',
  production_date: '', expiry_date: '', in_date: '', supplier: '',
  storage_condition: '', status: '在库', operator: '', remark: '',
})

const form = reactive(emptyForm())

function fetch(params) {
  return listReagents(params)
}
function onAdd() {
  Object.assign(form, emptyForm())
  editingId.value = null
  dialogVisible.value = true
}
function onEdit(row) {
  Object.assign(form, emptyForm(), row)
  editingId.value = row.id
  dialogVisible.value = true
}
async function onSubmit() {
  submitting.value = true
  try {
    if (editingId.value) {
      await updateReagent(editingId.value, { ...form })
    } else {
      await createReagent({ ...form })
    }
    ElMessage.success('已保存')
    dialogVisible.value = false
    crud.value?.refresh()
```

```
  } catch (e) {
    ElMessage.error('保存失败')
  } finally {
    submitting.value = false
  }
}
async function onDelete(row) {
  await ElMessageBox.confirm(`确认删除「${row.name}」？`, '提示', { type: 'warning' })
  await deleteReagent(row.id)
  ElMessage.success('已删除')
  crud.value?.refresh()
}
</script>

<style scoped>
.page {
  height: 100%;
}
</style>

========================================================================
# 文件: components/reagent/LibraryTabs.vue  (D:\workbuddyprojects\网页版-生免速查工具
\frontend\src\components\reagent\LibraryTabs.vue)
========================================================================

<template>
  <div class="library-tabs">
    <span class="label">责任库：</span>
    <el-radio-group v-model="selected" size="small" @change="onRadioChange">
      <el-radio-button v-for="lib in libs" :key="lib" :value="lib">{{ lib }}</el-radio-button>
    </el-radio-group>
    <span class="hint">（生化凝血 / 免疫 分开管理，两人各管一类）</span>
  </div>
</template>

<script setup>
import { ref, watch, onMounted } from 'vue'
import { useReagentStore, LIBRARIES } from '../../store/reagent'

const reagentStore = useReagentStore()
const libs = LIBRARIES
const selected = ref(reagentStore.library)

// 外部切换（如其他页面修改了 store）时同步回来
watch(() => reagentStore.library, (val) => {
  if (val !== selected.value) selected.value = val
})

function onRadioChange(val) {
  reagentStore.setLibrary(val)
  emit('change', val)
}
const emit = defineEmits(['change'])
</script>
```

```
<style scoped>
.library-tabs { display: flex; align-items: center; gap: 8px; margin: 4px 0 10px; flex-wrap: wrap; }
.library-tabs .label { font-size: 13px; color: #475569; font-weight: 600; }
.library-tabs .hint { font-size: 12px; color: #94a3b8; }
</style>

================================================================================
# 文件: api/reagent.js  (D:\workbuddyprojects\网页版-生免速查工具\frontend\src\api\reagent.js)
================================================================================

/**
 * 试剂管理模块 API 调用
 */

import request from '../utils/request'

// ── 试剂目录 ──
export function listReagentItems(params) {
  return request.get('/api/v1/reagent/items', { params })
}

export function getReagentItemCounts(params = {}) {
  return request.get('/api/v1/reagent/items/counts', { params })
}

// 分页拉全试剂目录（后端 page_size 上限 200，超 200 触发 422）。
// 用于在库存/月消耗等页面一次性拿到全部试剂映射，避免 [object Object] 报错。
// 默认包含停用项（show_inactive=true），保证盘库详情/打印的名称映射完整。
export async function listAllReagentItems(extra = {}) {
  let page = 1
  const all = []
  while (true) {
    const r = await listReagentItems({ page, page_size: 200, show_inactive: true, ...extra })
    all.push(...(r.items || []))
    if (all.length >= (r.total || 0) || (r.items || []).length === 0) break
    page += 1
    if (page > 100) break
  }
  return all
}

// 盘库/订购录入模板：按项目、仪器家族、质控品分组（含实时库存）
export function getReagentTemplate(params) {
  return request.get('/api/v1/reagent/template', { params })
}

export function getReagentItem(id) {
  return request.get(`/api/v1/reagent/items/${id}`)
}

export function createReagentItem(data) {
  return request.post('/api/v1/reagent/items', data)
}
```

```
export function updateReagentItem(id, data) {
  return request.put(`/api/v1/reagent/items/${id}`, data)
}

export function deleteReagentItem(id) {
  return request.delete(`/api/v1/reagent/items/${id}`)
}

export function importReagentFromExcel(formData) {
  return request.post('/api/v1/reagent/items/_import-excel', formData, {
    headers: { 'Content-Type': 'multipart/form-data' },
  })
}

// ── 实时库存 ──
export function listReagentStock(params) {
  return request.get('/api/v1/reagent/stock', { params })
}

// ── 盘库 ──
export function listInventoryChecks(params) {
  return request.get('/api/v1/reagent/inventory-checks', { params })
}

export function getInventoryCheck(id) {
  return request.get(`/api/v1/reagent/inventory-checks/${id}`)
}

export function createInventoryCheck(data) {
  return request.post('/api/v1/reagent/inventory-checks', data)
}

export function deleteInventoryCheck(id) {
  return request.delete(`/api/v1/reagent/inventory-checks/${id}`)
}

// ── 订购 ──
export function listReagentOrders(params) {
  return request.get('/api/v1/reagent/orders', { params })
}

export function getReagentOrder(id) {
  return request.get(`/api/v1/reagent/orders/${id}`)
}

export function getNextOrderNo() {
  return request.get('/api/v1/reagent/orders/_next-no')
}

export function createReagentOrder(data) {
```

```
  return request.post('/api/v1/reagent/orders', data)
}

export function updateReagentOrder(id, data) {
  return request.put(`/api/v1/reagent/orders/${id}`, data)
}

export function confirmReagentOrder(id) {
  return request.post(`/api/v1/reagent/orders/${id}/confirm`)
}

export function exportOrderForm(id) {
  return request.get(`/api/v1/reagent/orders/${id}/export-form`, { responseType: 'blob' })
}

// ── 到货接收 ──
export function listReagentReceivings(params) {
  return request.get('/api/v1/reagent/receivings', { params })
}

export function getReagentReceiving(id) {
  return request.get(`/api/v1/reagent/receivings/${id}`)
}

export function createReagentReceiving(data) {
  return request.post('/api/v1/reagent/receivings', data)
}

export function updateReagentReceiving(id, data) {
  return request.put(`/api/v1/reagent/receivings/${id}`, data)
}

export function confirmReagentReceiving(id) {
  return request.post(`/api/v1/reagent/receivings/${id}/confirm`)
}
export function deleteReagentReceiving(id) {
  return request.delete(`/api/v1/reagent/receivings/${id}`)
}

// ── 月消耗 ──
export function listReagentConsumption(params) {
  return request.get('/api/v1/reagent/consumption', { params })
}

export function calculateConsumption(yearMonth) {
  return request.post(`/api/v1/reagent/consumption/_calculate?year_month=${yearMonth}`)
}

// ── 项目↔试剂 / 仪器↔耗材 关联 ──
export function listTestItemReagents(params) {
```

```
  return request.get('/api/v1/reagent/associations/test-items', { params })
}
export function createTestItemReagent(data) {
  return request.post('/api/v1/reagent/associations/test-items', data)
}
export function updateTestItemReagent(id, data) {
  return request.put(`/api/v1/reagent/associations/test-items/${id}`, data)
}
export function deleteTestItemReagent(id) {
  return request.delete(`/api/v1/reagent/associations/test-items/${id}`)
}
export function listInstrumentReagents(params) {
  return request.get('/api/v1/reagent/associations/instruments', { params })
}
export function createInstrumentReagent(data) {
  return request.post('/api/v1/reagent/associations/instruments', data)
}
export function updateInstrumentReagent(id, data) {
  return request.put(`/api/v1/reagent/associations/instruments/${id}`, data)
}
export function deleteInstrumentReagent(id) {
  return request.delete(`/api/v1/reagent/associations/instruments/${id}`)
}
export function autoMatchAssociations(reset = false) {
  return request.post(`/api/v1/reagent/associations/_auto-match?reset=${reset}`)
}

// 待关联建议：返回未关联但可匹配 / 真正无候选的试剂，供人工审核采纳
export function getAssociationSuggestions(params = {}) {
  return request.get('/api/v1/reagent/associations/suggestions', { params })
}

==========================================================================
# 文件: api/reagents.js  (D:\workbuddyprojects\网页版-生免速查工具\frontend\src\api\reagents.js)
==========================================================================

import request from '../utils/request'

export function listReagents(params) {
  return request.get('/api/v1/reagents', { params })
}
export function createReagent(data) {
  return request.post('/api/v1/reagents', data)
}
export function updateReagent(id, data) {
  return request.put(`/api/v1/reagents/${id}`, data)
}
export function deleteReagent(id) {
  return request.delete(`/api/v1/reagents/${id}`)
}

==========================================================================
# 文件: store/reagent.js  (D:\workbuddyprojects\网页版-生免速查工具\frontend\src\store\reagent.js)
==========================================================================

import { defineStore } from 'pinia'

// 试剂管理「责任库」全局状态：生化凝血 / 免疫 两类分管。
```

```
// 各试剂页面共享当前选中的责任库，刷新后仍记住选择。
const STORAGE_KEY = 'reagent_library'

export const LIBRARIES = ['生化凝血', '免疫']

export const useReagentStore = defineStore('reagent', {
  state: () => ({
    library: localStorage.getItem(STORAGE_KEY) || '生化凝血',
  }),
  actions: {
    setLibrary(lib) {
      if (!LIBRARIES.includes(lib)) lib = '生化凝血'
      this.library = lib
      localStorage.setItem(STORAGE_KEY, lib)
    },
  },
})
```